

Introduction to Data Science and Engineering

- R programming



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

RB Luo / 羅銳邦

School of Computing and Data Science
University of Hong Kong

Some slides retrofitted by Michael Wu

The two sides of R programming

It's easy to learn



It's difficult to master





Why R?

- **R is not a programming language like C or Java.** It was not created by software engineers for software development. Instead, it was developed by statisticians as an interactive environment for data analysis
- **The interactivity is an indispensable feature in data science** because, as you will soon learn, the ability to quickly explore data is a necessity for success in this field
- However, like in other programming languages, **you can save your work as scripts that can be easily executed at any moment.** These scripts serve as a record of the analysis you performed, a key feature that facilitates reproducible work
- **If you are an expert programmer, you should not expect R to follow** the conventions you are used to since you will be disappointed
- **If you are patient, you will come to appreciate the unequal power of R** when it comes to data analysis and, **specifically, data visualization**



Other attractive features of R

- R is free and open source
- It runs on all major platforms: Windows, MacOS, UNIX/Linux
- Scripts and data objects can be shared seamlessly across platforms
- There is a large, growing, and active community of R users and, as a result, there are numerous resources for learning and asking questions
- It is easy for others to contribute add-ons which enables developers to share packages of new data science domains, including finance, ecology, molecular biology, social sciences, geography ...

Nevertheless,
MS Excel still
wins

"I'm a data scientist" starterpack

buzzword connoisseur: "blockchain",
"deep learning", "neural network", "AI"

Degree in:
* computer science ✖
* statistics ✖
* applied math ✖
* business analytics ✔

OR * did a \$20K bootcamp
* 2 coursera certifications

skills:
makes pretty graphs ✔

R ✖
Python ✖
SQL ✖
MS Excel ✔

constantly posts on this:



wrote a python tutorial on here:



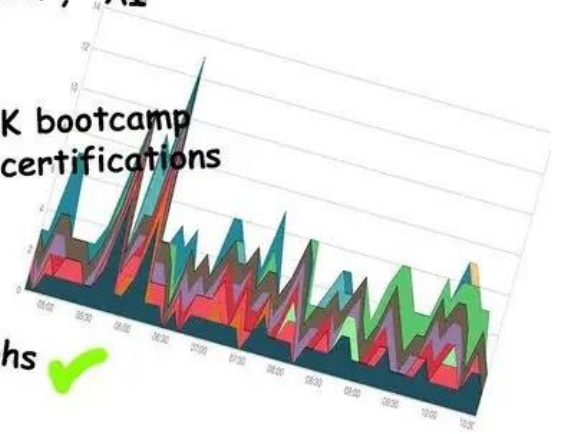
Medium

"how do i do machine learning"



stackoverflow

"how can i get a phd
salary without a phd?"

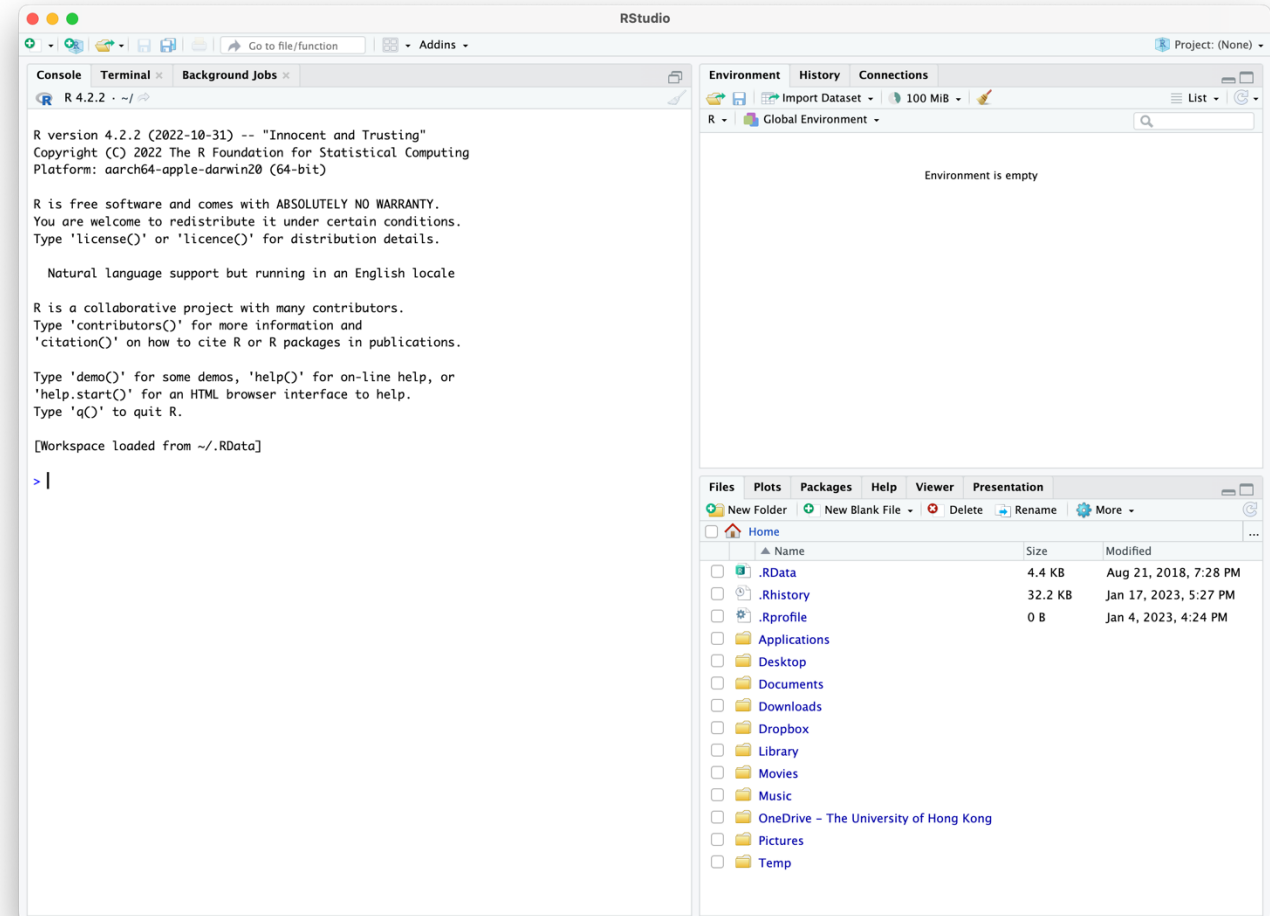


RStudio

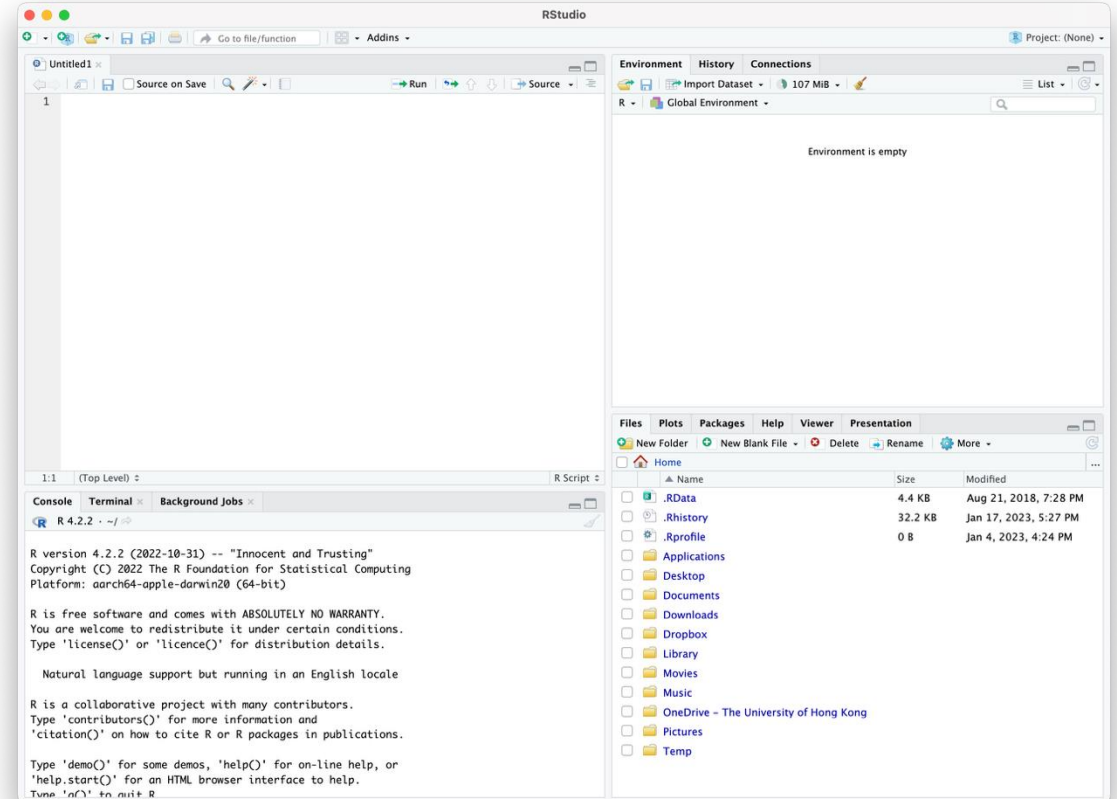
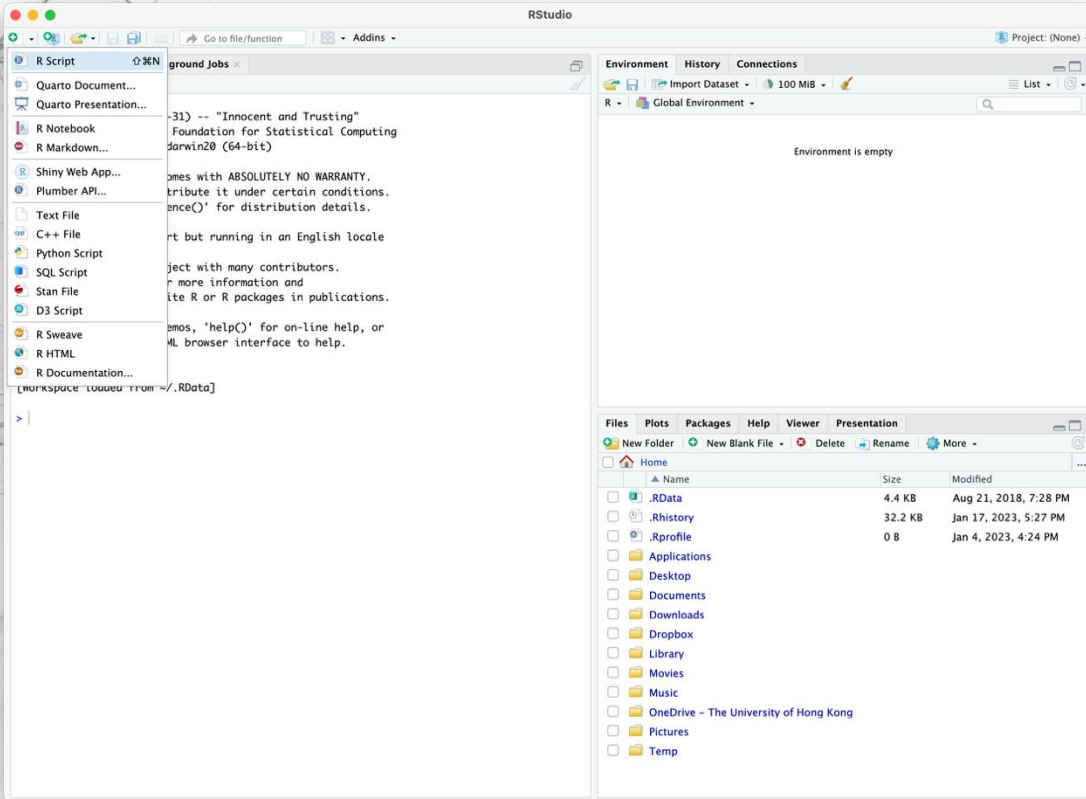


The panes

- RStudio will be our launching pad for data science projects. It not only provides an editor for us to create and edit our scripts but also provides many other useful tools.

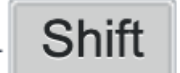


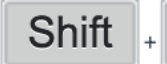
Create a new script



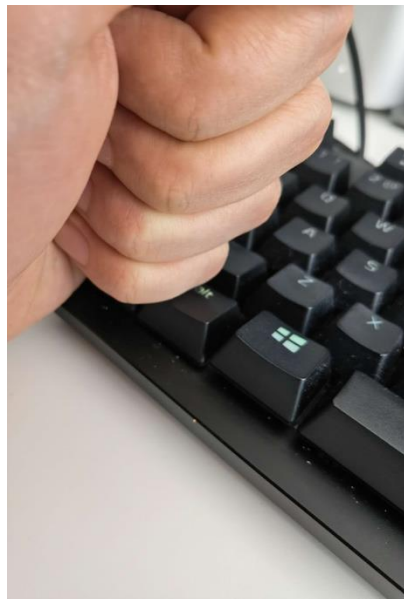
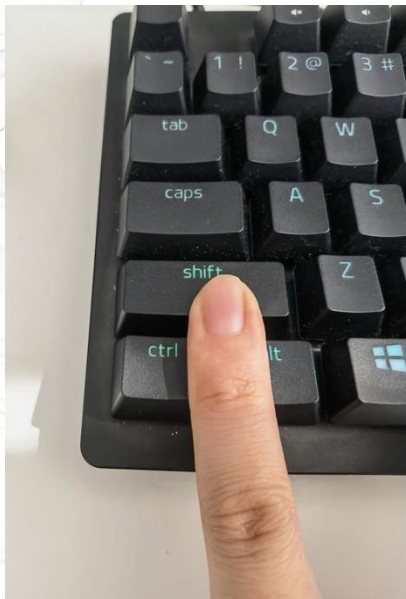
Use less mouse and start using keyboard shortcut

Create a new script

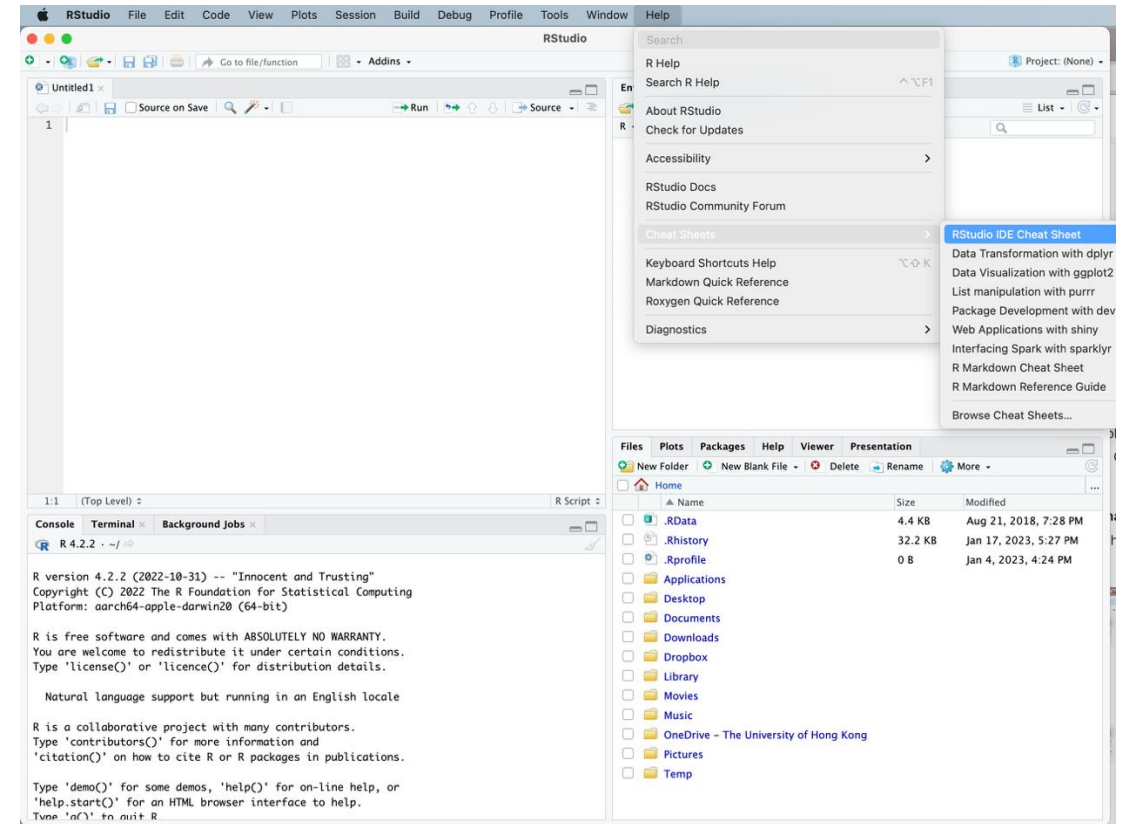
Windows  +  + 

Mac  +  + 

What sophisticated
data scientists do



Check the cheatsheet



Install R and RStudio

- Install R (the programming language) at: <https://cran.r-project.org/>



CRAN
[Mirrors](#)
[What's new?](#)
[Search](#)
[CRAN Team](#)

About R
[R Homepage](#)

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages,
Windows and Mac users most likely want one of these versions of R:

- [Download R for Linux](#) ([Debian](#), [Fedora/Redhat](#), [Ubuntu](#))
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

- Special notes for Mac users

For Apple silicon (M1,2,..) Macs:

[R-4.5.1-arm64.pkg](#)

SHA1-hash: 0db802faf0e544168794a6d648c73a48c2b51a5d
(ca. 97MB, notarized and signed)

For older Intel Macs:

[R-4.5.1-x86_64.pkg](#)

SHA1-hash: 5384a1b3458a28030fc043e64c113e3af40f4c58
(ca. 100MB, notarized and signed)



Install R and RStudio

- Install R Studio (Integrated Development Environment, IDE) at: <https://posit.co/downloads/>

DOWNLOAD

RStudio IDE

The most popular coding environment for R, built with love by Posit.

Used by millions of people weekly, the RStudio integrated development environment (IDE) is a set of tools built to help you be more productive with R and Python. It includes a console, syntax-highlighting editor that supports direct code execution. It also features tools for plotting, viewing history, debugging and managing your workspace.

If you're a professional data scientist and want guidance on adopting open source in your organization, don't hesitate to [book a call with us.](#)

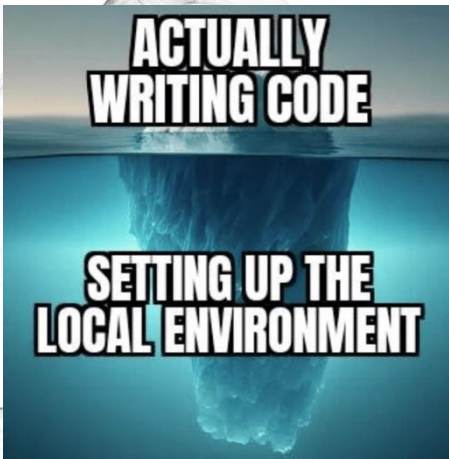
DOWNLOAD RSTUDIO

DOWNLOAD RSTUDIO SERVER



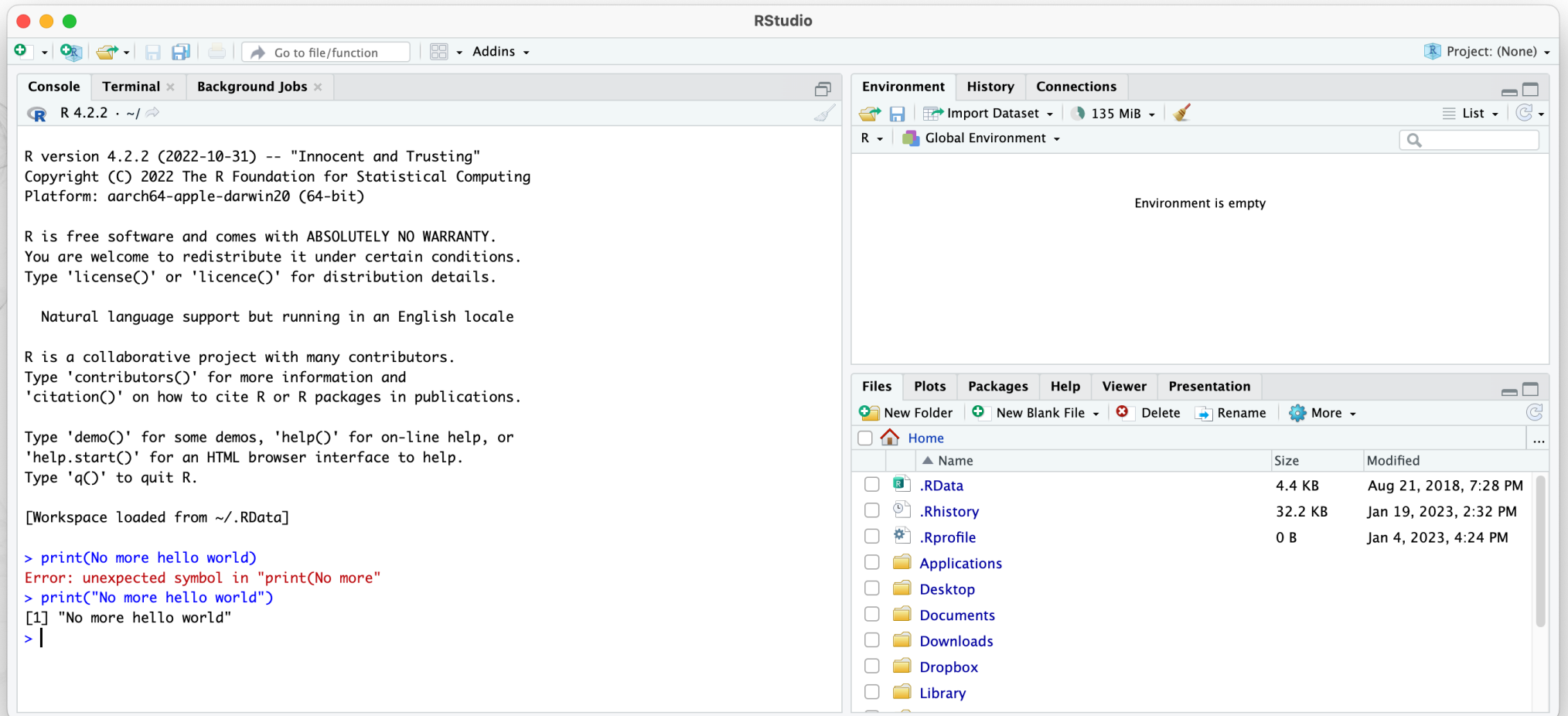
Install R and RStudio

- If you got into problem installing R or RStudio
 - Google the error messages (or Bing, Baidu, ChatGPT) first, try fixing the problems yourself
 - Show your problem to your peers and friends, see if they had run into the same problem
 - Ask TAs for help (email, f2f, zoom)
- More generally, the right mindset to have with running environment setup and management
 - Good chefs use their own cookware
 - Knowing about operating systems and computer hardware is not others' job



First ritual first

Do it in the console



The image shows the RStudio interface. The console on the left displays the R version 4.2.2 (2022-10-31) -- "Innocent and Trusting" and the copyright notice. It also shows the license information and the workspace loaded from ~/.RData. The file explorer on the right shows the Home directory with files .RData, .Rhistory, and .Rprofile, and folders Applications, Desktop, Documents, Downloads, Dropbox, and Library.

```
R version 4.2.2 (2022-10-31) -- "Innocent and Trusting"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: aarch64-apple-darwin20 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

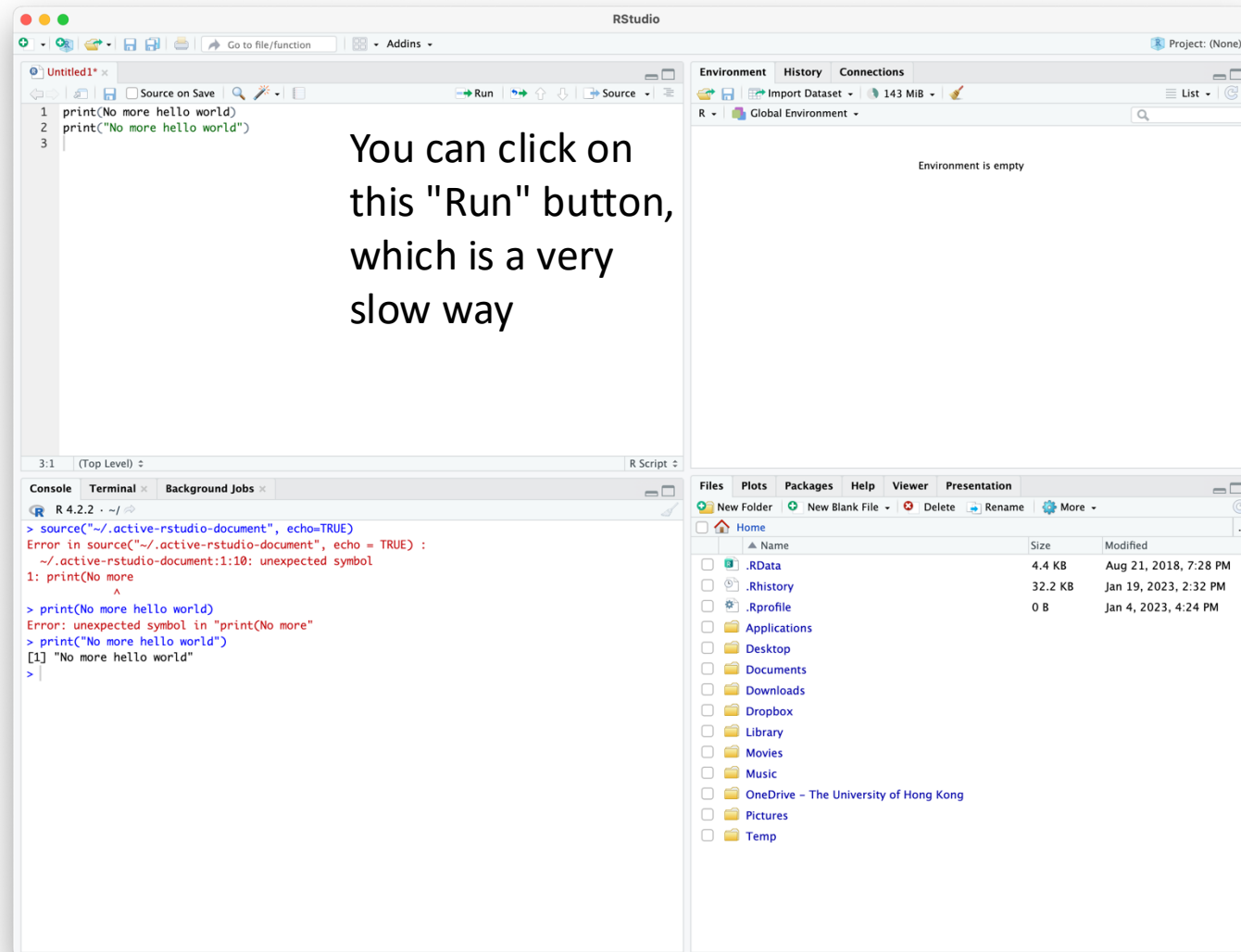
[Workspace loaded from ~/.RData]

> print(No more hello world)
Error: unexpected symbol in "print(No more"
> print("No more hello world")
[1] "No more hello world"
> |
```

Name	Size	Modified
.RData	4.4 KB	Aug 21, 2018, 7:28 PM
.Rhistory	32.2 KB	Jan 19, 2023, 2:32 PM
.Rprofile	0 B	Jan 4, 2023, 4:24 PM
Applications		
Desktop		
Documents		
Downloads		
Dropbox		
Library		

First ritual first

Run commands from the script editor



You can click on
this "Run" button,
which is a very
slow way

Run the current line:
Mac: Cmd+Enter
Win: Ctrl+Enter

Run the whole script:
Mac: Cmd+Shift+Enter
Win: Ctrl+Shift+Enter

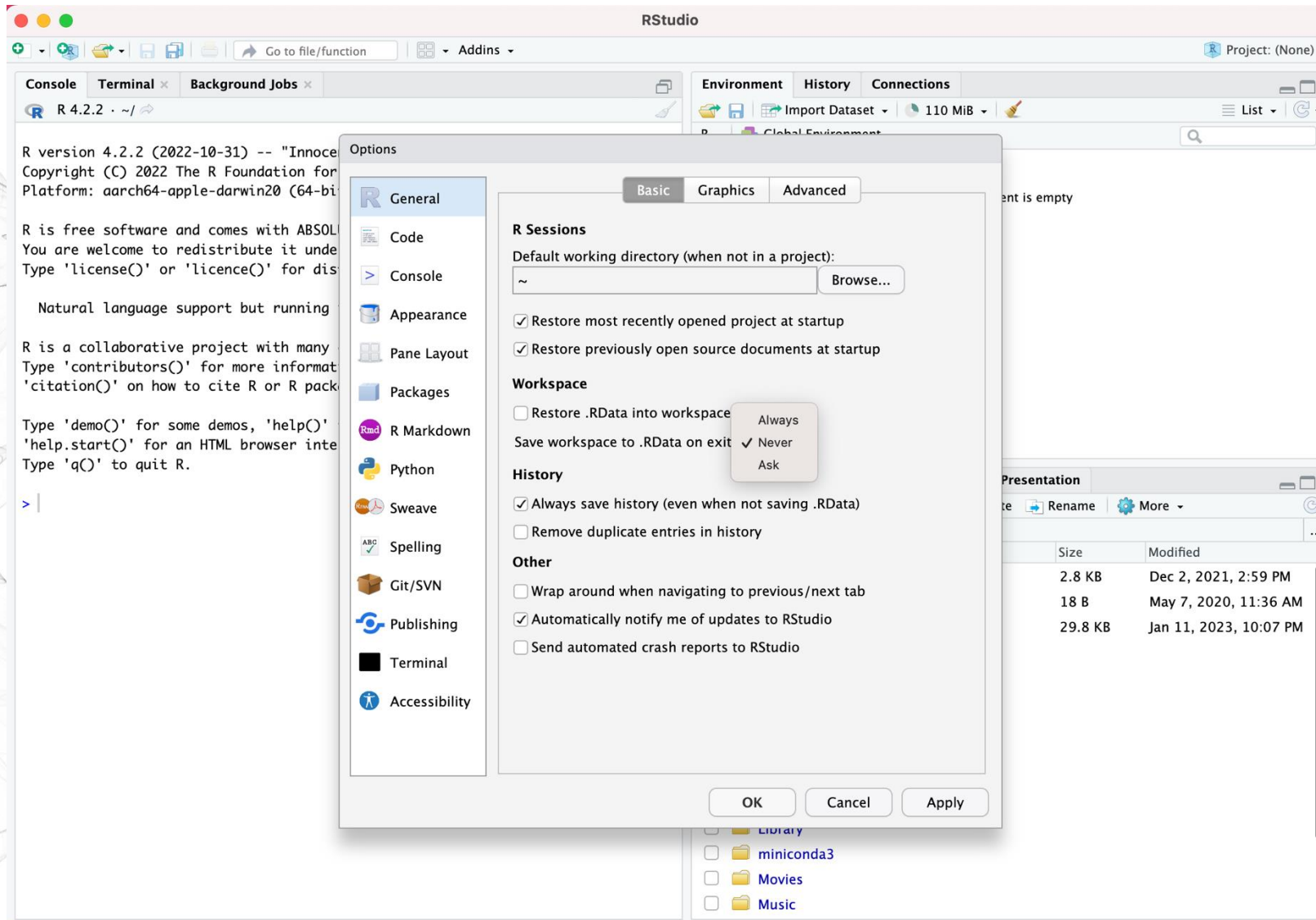
Use shortcuts

Keyboard Shortcut Quick Reference

Category	Shortcut	Action		
Tabs	^T	Switch to Tab in 20		
	^N	Next Tab		
	^P	Previous Tab		
	^F11	First Tab		
	^F12	Last Tab		
	^G	Find Next		
	^G	Find Previous		
	^H	Replace and Find		
	^T	Go To File/Function...		
	^G	Go to Line...		
	^J	Jump To...		
	^E	Expand To Matching		
Panes	^1	Move Focus to Source		
	^2	Move Focus to Console		
	^3	Move Focus to Help		
	^4	Show History		
	^5	Show Files		
	^6	Show Plots		
	^7	Show Packages		
	^8	Show Environment		
	^9	Show Viewer		
	^F1	Show Vcs		
	^F2	Show Build		
	^F8	Show Presentation		
Files	^F6	Show Find in Files		
	^F7	Add Source Column		
	^0	Show All Panes		
	^S	Save		
	^O	Open		
	^C	Close		
	^W	Close All		
	^D	Save All		
	^Z	Undo		
	^Y	Redo		
	^X	Cut		
	^V	Paste		
Source Editor	^I	Insert Chunk		
	^R	Insert Section...		
	^X	Extract Function		
	^V	Extract Variable		
	^C	Comment/Uncomment Lines		
	^I	Reindent Lines		
	^/	Reflow Comment		
	^A	Reformat Code		
	^D	Show Diagnostics (Project)		
	^L	Collapse Fold		
	^L	Expand Fold		
	^O	Collapse All Folds		
Source Control	^D	Diff Files		
	^M	Commit...		
	^K	Compile PDF		
	^K	Preview		
	^K	Knit Document		
	^K	Render Document		
	^K	Run Document		
	^B	Install Package		
	^L	Load All		
	^E	Check Package		
	^T	Test Package		
	^D	Document		
Build	^K	Compile PDF		
	^K	Preview		
	^K	Knit Document		
	^K	Render Document		
	^K	Run Document		
	^B	Install Package		
	^L	Load All		
	^E	Check Package		
	^T	Test Package		
	^D	Document		
	Console	^L	Clear Console	
		^P	Popout Command History	
^R		New Terminal		
^M		Move Focus to Terminal		
^F11		Previous Terminal		
^F12		Next Terminal		
Terminal		^R	New Terminal	
		^M	Move Focus to Terminal	
		^F11	Previous Terminal	
		^F12	Next Terminal	
		Debug	^B	Breakpoint
			^D	Debug
	^F		Find	
	^G		Go	
	^H		Help	
	^I		Insert	
	^J		Jump	
	^K		Kill	
^L	Load			
^M	Move			
^N	New			
^O	Open			

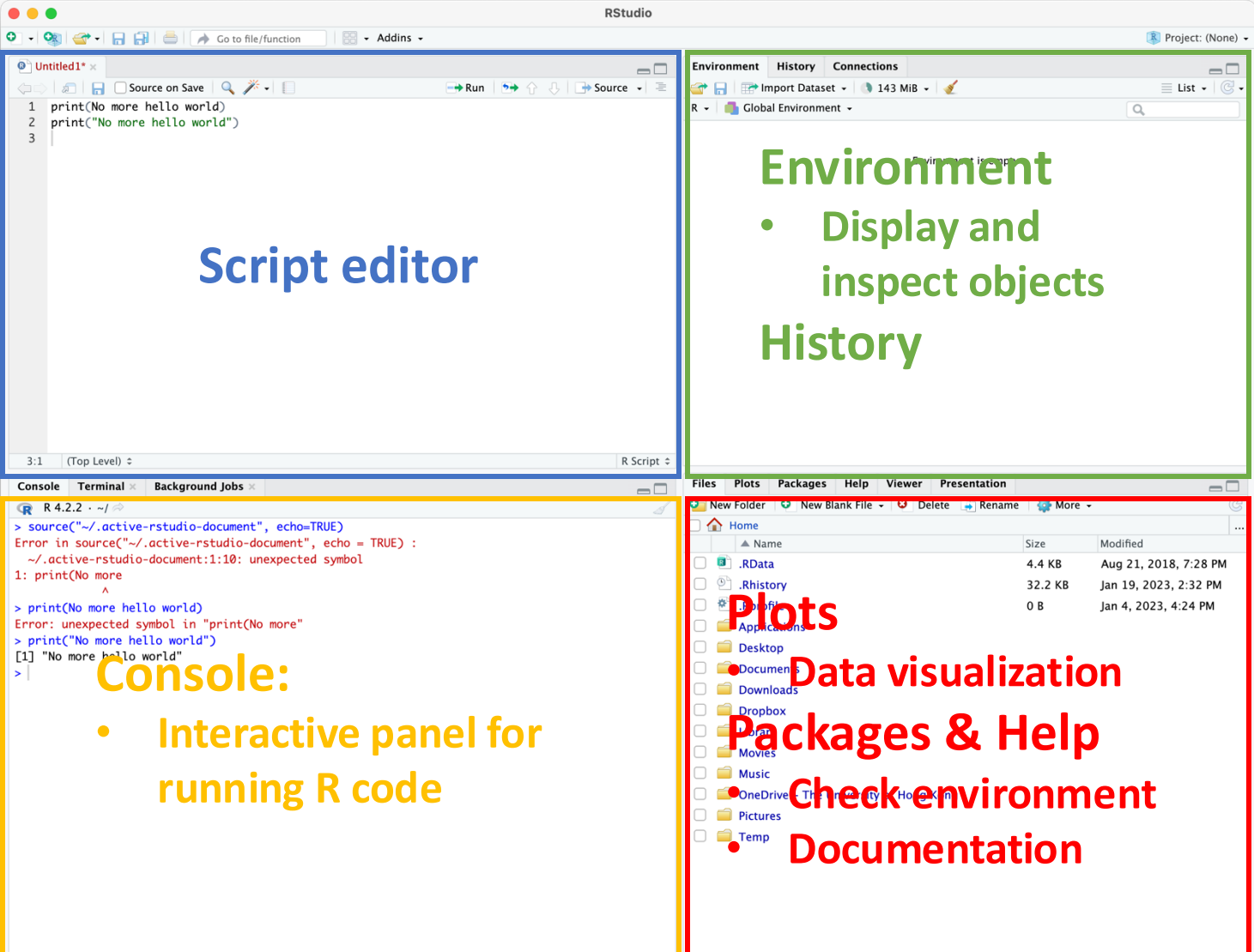
Change global options

In tools > global options > general



We suggest unclicking “Restore .Rdata into workspace” and change “Save workspace to .Rdata on exit” to “Never”. Can cause problem on exist when you have plenty of unneeded data in memory. Save your scripts frequently, and save your workspace when necessary.

Your workspace



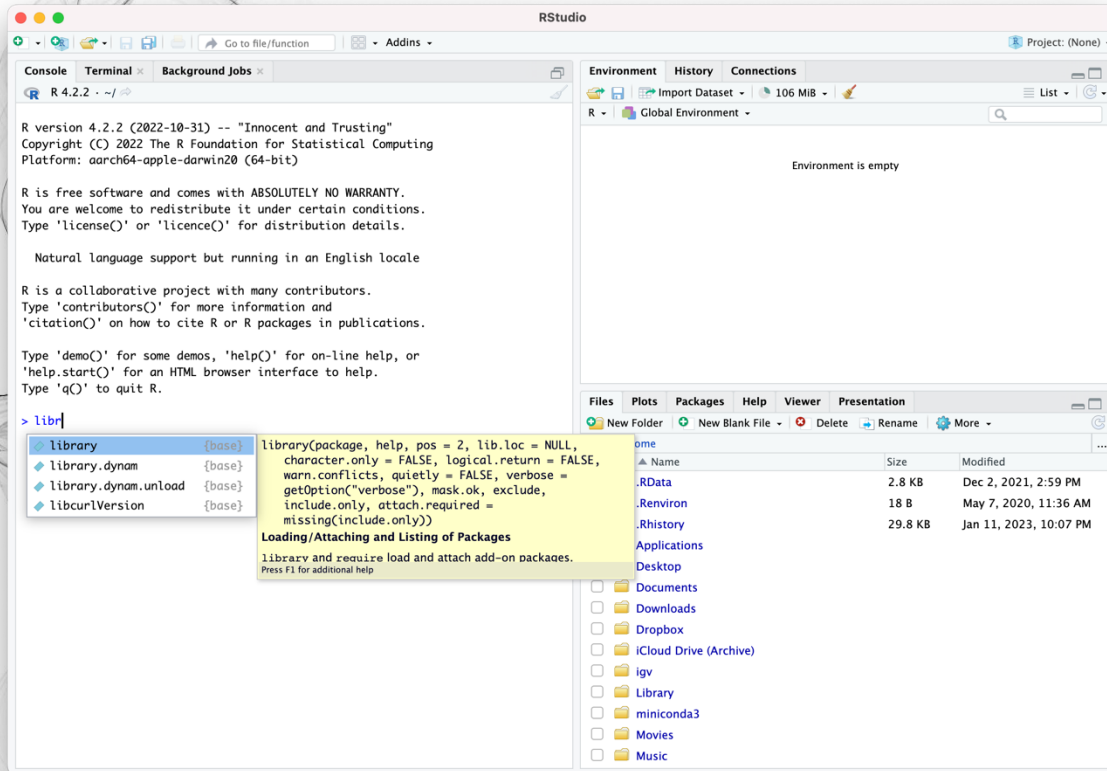
The image shows the RStudio interface with four main panels highlighted by colored boxes and labels:

- Script editor** (blue box): The top-left panel where R code is written. It contains the following code:

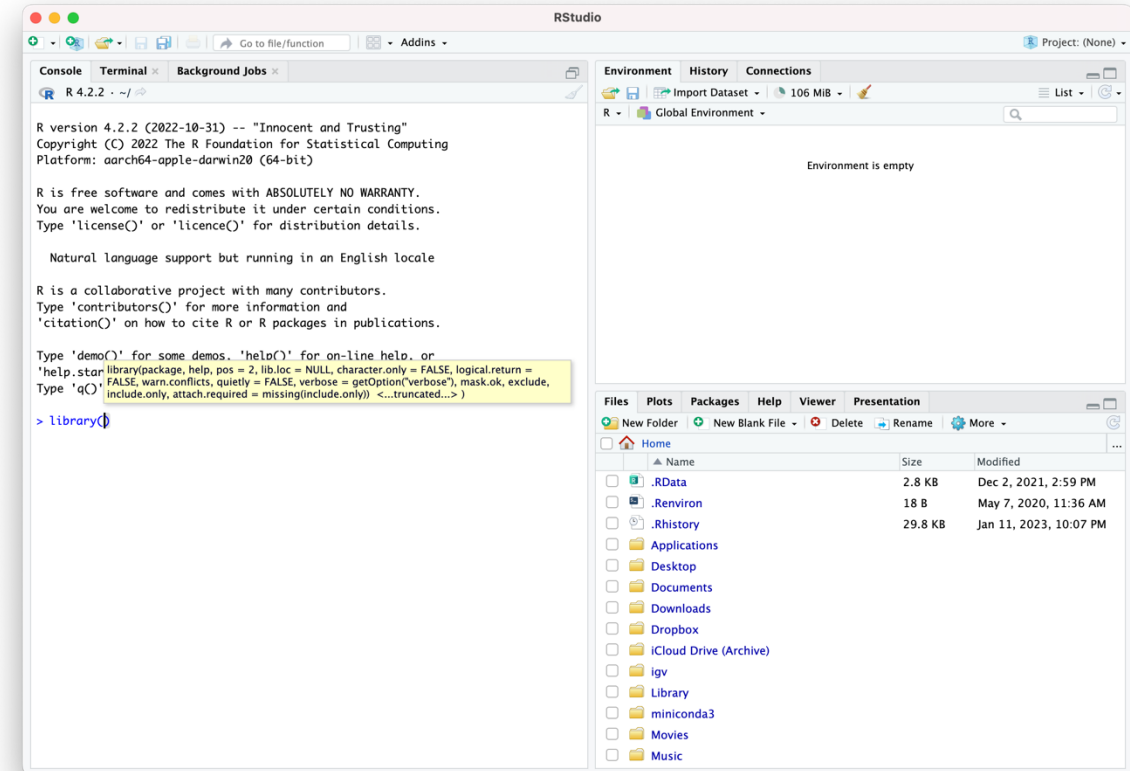
```
1 print("No more hello world")
2 print("No more hello world")
3
```
- Environment** (green box): The top-right panel, labeled "Environment" in green. It contains a bullet point: "Display and inspect objects". Below it, the word "History" is also in green.
- Console** (yellow box): The bottom-left panel, labeled "Console" in yellow. It shows the output of running the code in the script editor:

```
> source("~/active-rstudio-document", echo=TRUE)
Error in source("~/active-rstudio-document", echo = TRUE) :
  ~/active-rstudio-document:1:10: unexpected symbol
1: print("No more
  ^
> print("No more hello world")
Error: unexpected symbol in "print("No more"
> print("No more hello world")
[1] "No more hello world"
>
```
- Plots** (red box): The bottom-right panel, labeled "Plots" in red. It contains a list of items: "Data visualization", "Packages & Help", "Check environment", and "Documentation".

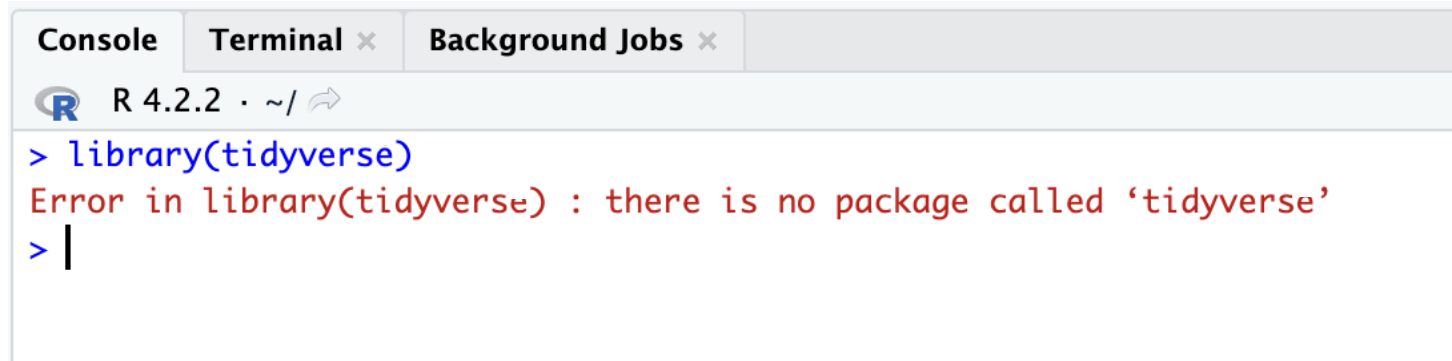
Load a package (learn autocompletion)



Press
tab



Load a package



The image shows a screenshot of the RStudio interface. At the top, there are three tabs: 'Console', 'Terminal', and 'Background Jobs'. The 'Console' tab is active. Below the tabs, the R logo and version 'R 4.2.2 · ~/ ' are displayed. The console shows the command `> library(tidyverse)` in blue. Below it, an error message is displayed in red: `Error in library(tidyverse) : there is no package called 'tidyverse'`. The prompt `> |` is visible at the bottom of the console.

```
Console Terminal x Background Jobs x
R 4.2.2 · ~/ 
> library(tidyverse)
Error in library(tidyverse) : there is no package called 'tidyverse'
> |
```

Standard procedures:

Install it with “`install.packages("tidyverse")`”

Also, please install the package “dslabs”

How to install multiple packages?

```
install.packages(c("tidyverse", "dslabs"))
```

But let’s do this to install a specific version of tidyverse library:

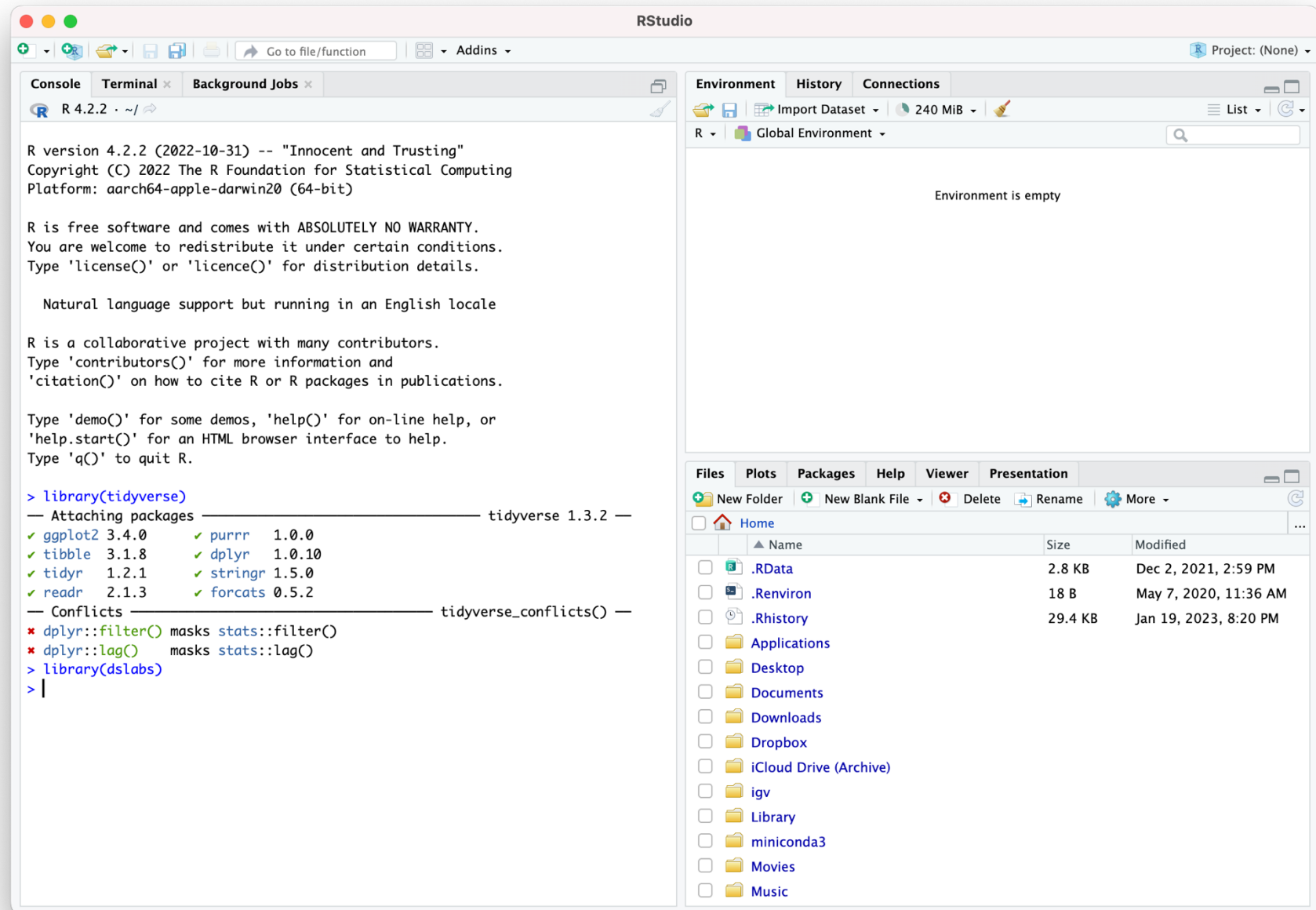
```
install.packages("remotes")
```

```
library(remotes)
```

```
install_version("tidyverse", "1.3.2")
```

Why?

Package loaded



The image shows the RStudio interface with the following components:

- Console:** Displays the R version (4.2.2), copyright information, and the output of the `library(tidyverse)` command. The output shows that several packages (ggplot2, purrr, tibble, dplyr, tidyr, stringr, readr, forcats) were successfully loaded. It also shows conflicts between `dplyr::filter()` and `stats::filter()`, and `dplyr::lag()` and `stats::lag()`.
- Environment:** Shows the Global Environment, which is currently empty.
- Files:** A file explorer showing the contents of the Home directory, including folders like .RData, .Renviron, .Rhistory, Applications, Desktop, Documents, Downloads, Dropbox, iCloud Drive (Archive), igv, Library, miniconda3, Movies, and Music.

```
R 4.2.2 · ~/   
  
R version 4.2.2 (2022-10-31) -- "Innocent and Trusting"  
Copyright (C) 2022 The R Foundation for Statistical Computing  
Platform: aarch64-apple-darwin20 (64-bit)  
  
R is free software and comes with ABSOLUTELY NO WARRANTY.  
You are welcome to redistribute it under certain conditions.  
Type 'license()' or 'licence()' for distribution details.  
  
Natural language support but running in an English locale  
  
R is a collaborative project with many contributors.  
Type 'contributors()' for more information and  
'citation()' on how to cite R or R packages in publications.  
  
Type 'demo()' for some demos, 'help()' for on-line help, or  
'help.start()' for an HTML browser interface to help.  
Type 'q()' to quit R.  
  
> library(tidyverse)   
— Attaching packages — tidyverse 1.3.2 —  
✓ ggplot2 3.4.0      ✓ purrr  1.0.0  
✓ tibble 3.1.8      ✓ dplyr  1.0.10  
✓ tidyr  1.2.1      ✓ stringr 1.5.0  
✓ readr  2.1.3      ✓ forcats 0.5.2  
— Conflicts — tidyverse_conflicts() —  
✖ dplyr::filter() masks stats::filter()  
✖ dplyr::lag() masks stats::lag()  
> library(dslabs)  
> |
```

Keep track of the packages you've installed

Files

Plots

Packages

Help

Viewer

Presentation

Install

Update

Name	Description	Version		
System Library				
☐ animation	A Gallery of Animations in Statistics and Utilities to Create Animations	2.7		
☐ annotate	Annotation for microarrays	1.76.0		
☐ AnnotationDbi	Manipulation of SQLite-based annotations in Bioconductor	1.60.0		
☐ askpass	Safe Password Entry for R, Git, and SSH	1.1		
☐ assertthat	Easy Pre and Post Assertions	0.2.1		
☐ backports	Reimplementations of Functions Introduced Since R-3.0.0	1.4.1		
☒ base	The R Base Package	4.2.2		
☐ base64enc	Tools for base64 encoding	0.1-3		
☐ Biobase	Biobase: Base functions for Bioconductor	2.58.0		
☐ BiocGenerics	S4 generic functions used in Bioconductor	0.44.0		
☐ BiocManager	Access the Bioconductor Project Package Repository	1.30.19		
☐ BiocVersion	Set the appropriate version of Bioconductor packages	3.16.0		
☐ Biostrings	Efficient manipulation of biological strings	2.66.0		



The very basics of R

Objects

Suppose we gonna solve a quadratic equation

$$x^2 + x - 1 = 0,$$

The quadratic formula gives us the solutions

$$\frac{-b - \sqrt{b^2 - 4ac}}{2a} \quad \text{and} \quad \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

First, we define the three variables

```
Console Terminal x Background Jobs x
R 4.2.2 · ~/
> a <- 1
> b <- 1
> c <- -1
>
```

Using = instead of <- for assignment



We use "<-" to **assign** values to the variables.

Compare to python:

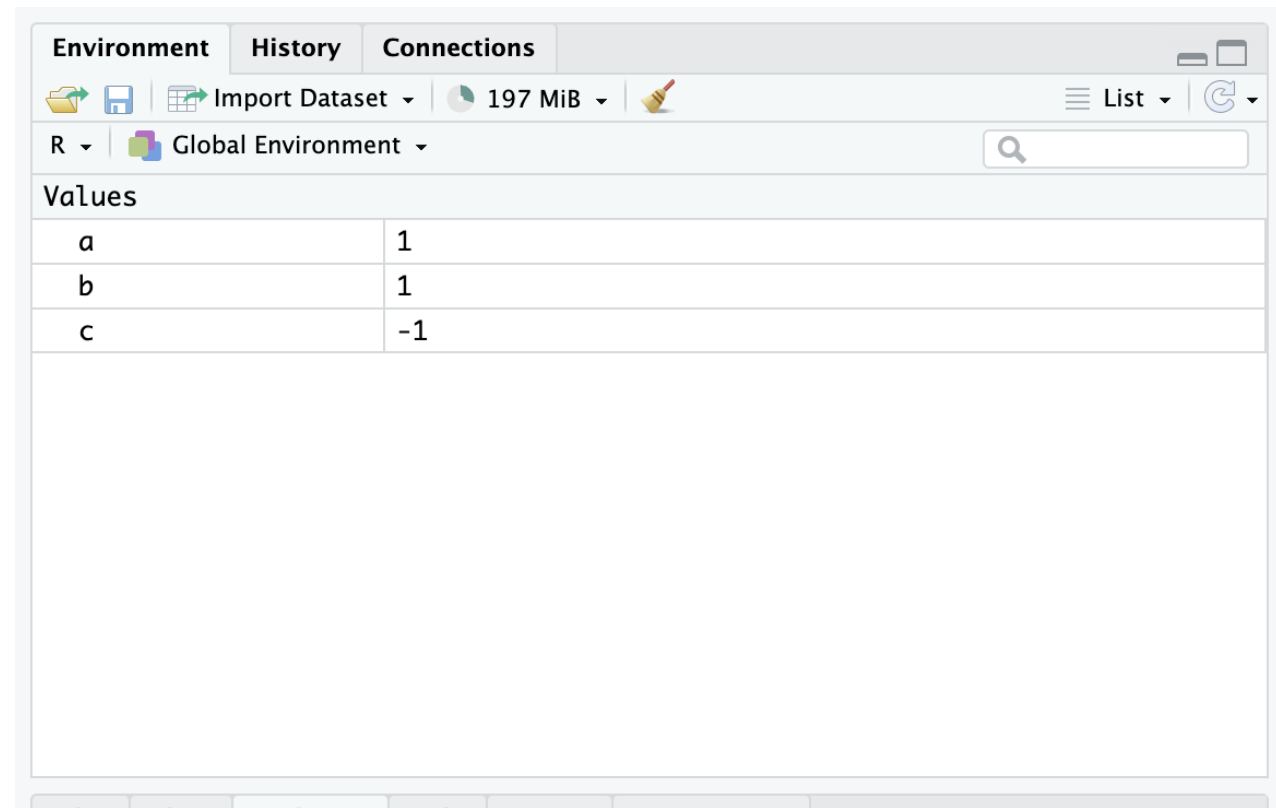
```
a = 1
b = 1
c = -1
```

Actually "=" also works in R, but not recommended. Check more details at

<https://blog.revolutionanalytics.com/2008/12/use-equals-or-arrow-for-assignment.html>

Objects

After defining them, they should appear in “Environment” page



Environment	
History	
Connections	
197 MiB	
List	
R	
Global Environment	
Values	
a	1
b	1
c	-1



Objects

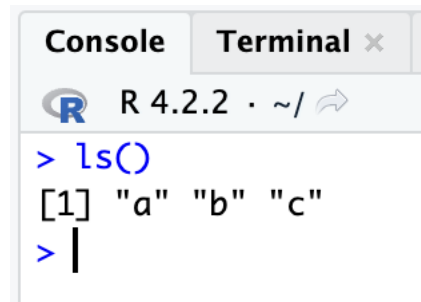
Print the variables

```
> a  
[1] 1  
> b  
[1] 1  
> c  
[1] -1  
> print(a)  
[1] 1  
> print(b)  
[1] 1  
> print(c)  
[1] -1  
>
```

We use the term *object* to describe stuff that is stored in R. Variables are examples, but objects can also be more complicated entities such as functions and data frame which are described later.

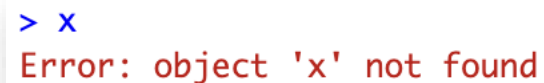
The workspace

As we define objects in the console, we are actually changing the workspace. You can see all the variables saved in your workspace by typing:



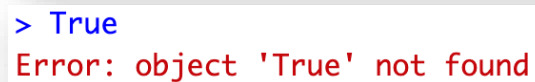
A screenshot of the R console interface. The title bar shows 'Console' and 'Terminal x'. Below the title bar, it says 'R 4.2.2 · ~/'. The prompt is '>'. The user has entered 'ls()' and the output is '[1] "a" "b" "c"'. The prompt is now '> |'.

"ls" is a function, you need a bracket to "call" it. What if we missed the ()? Give it a try



A screenshot of the R console showing an error. The prompt is '>'. The user has entered 'x' and the output is 'Error: object 'x' not found'.

Visiting an undefined object causes error



A screenshot of the R console showing an error. The prompt is '>'. The user has entered 'True' and the output is 'Error: object 'True' not found'.

Case-sensitive: This $\neq$ THIS $\neq$ this
TRUE is the logical value, True is not



The workspace

```
> (-b + sqrt(b^2 - 4*a*c) ) / ( 2*a )  
[1] 0.618034  
> (-b - sqrt(b^2 - 4*a*c) ) / ( 2*a )  
[1] -1.618034
```

In Python, square root function is called as “import math; math.sqrt(x)”

In R, sqrt is a function in 'stats'. 'stats' is imported by default so that sqrt can be called directly without loading or importing anything

A list of R built-in functions is at <https://www.javatpoint.com/r-built-in-functions>

Variable names

- Bad examples

- $a \leftarrow (-b + \sqrt{b^2 - 4ac}) / (2a)$
- $b \leftarrow (-b - \sqrt{b^2 - 4ac}) / (2a)$

- Good examples

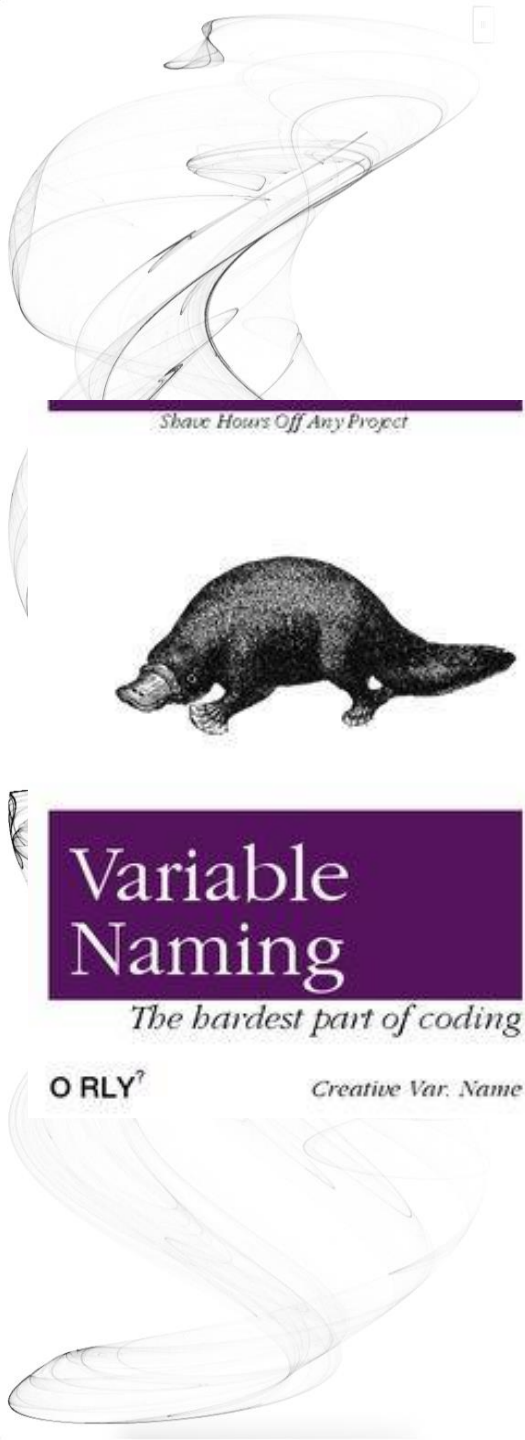
- `solution_1 <- (-b + sqrt(b^2 - 4*a*c)) / (2*a)`
- `solution_2 <- (-b - sqrt(b^2 - 4*a*c)) / (2*a)`

- VariableNameAsAComment style

- `OnJan15thIn2024WeGatheredAtRHTAndWitnessedSomeRidiculousPie`

- General recommendation: `snake_case`

- Be descriptive
- all lower case
- use_underscore_not_space



Functions

Let's look at another built-in function

```
> log(10)
[1] 2.302585
> log(a)
[1] 0
>
```

Here, unlike `ls`, most functions require one or more *arguments*
To find out what arguments a function expects, use:

```
Console Terminal x
R 4.2.2 · ~/
> help("log")
> ?log
> |
```



The screenshot shows the R Documentation window for the topic 'Logarithms and Exponentials'. The window has a menu bar with 'Files', 'Plots', 'Packages', 'Help', 'Viewer', and 'Presentation'. Below the menu bar is a search bar and a 'Find in Topic' button. The main content area is titled 'Logarithms and Exponentials' and includes a 'Description' section, a 'Usage' section, and an 'Arguments' section. The 'Description' section explains that `log` computes logarithms, with `log10` for common (base 10) and `log2` for binary (base 2) logarithms. It also mentions `log1p` for $\log(1+x)$ and `exp` for the exponential function. The 'Usage' section lists the functions `log(x, base = exp(1))`, `logb(x, base = exp(1))`, `log10(x)`, `log2(x)`, `log1p(x)`, `exp(x)`, and `expm1(x)`. The 'Arguments' section describes the parameters `x` (a numeric or complex vector) and `base` (a positive or complex number, defaulting to $e = \exp(1)$).

log {base} R Documentation

Logarithms and Exponentials

Description

`log` computes logarithms, by default natural logarithms, `log10` computes common (i.e., base 10) logarithms, and `log2` computes binary (i.e., base 2) logarithms. The general form `log(x, base)` computes logarithms with base `base`.

`log1p(x)` computes $\log(1+x)$ accurately also for $|x| \ll 1$.

`exp` computes the exponential function.

`expm1(x)` computes $\exp(x) - 1$ accurately also for $|x| \ll 1$.

Usage

```
log(x, base = exp(1))
logb(x, base = exp(1))
log10(x)
log2(x)

log1p(x)

exp(x)
expm1(x)
```

Arguments

x a numeric or complex vector.

base a positive or complex number: the base with respect to which logarithms are computed. Defaults to $e = \exp(1)$.

Functions

Files Plots Packages Help Viewer Presentation

R: Logarithms and Exponentials Find in Topic

log {base} R Documentation

Logarithms and Exponentials

Description

log computes logarithms, by default natural logarithms, log10 computes common (i.e., base 10) logarithms, and log2 computes binary (i.e., base 2) logarithms. The general form `log(x, base)` computes logarithms with base base.

`log1p(x)` computes $\log(1 + x)$ accurately also for $|x| \ll 1$.

`exp` computes the exponential function.

`expm1(x)` computes $\exp(x) - 1$ accurately also for $|x| \ll 1$.

Usage

```
log(x, base = exp(1))
logb(x, base = exp(1))
log10(x)
log2(x)

log1p(x)

exp(x)
expm1(x)
```

Arguments

x a numeric or complex vector.

base a positive or complex number: the base with respect to which logarithms are computed. Defaults to $e = \exp(1)$.

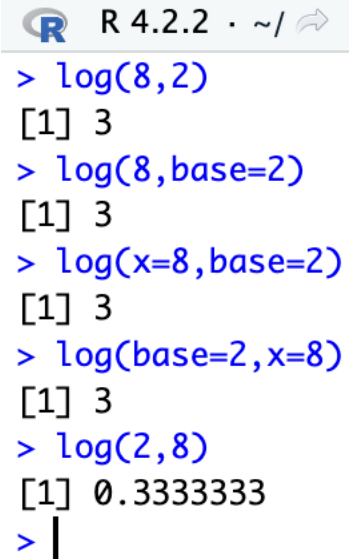
The help manual of log shows that log needs x and base to run

However, some arguments are required, and others are optional

You can determine which arguments are optional by noting in the help document that a default value is assigned with =

Defining these is optional. For example, the base of the function log defaults to base = exp(1), making log the natural log by default

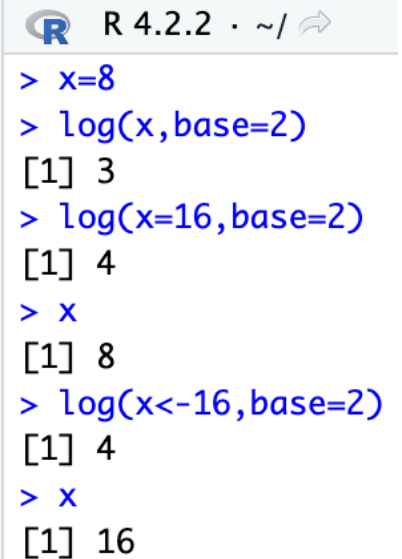
Functions



```
R 4.2.2 · ~/   
> log(8,2)  
[1] 3  
> log(8,base=2)  
[1] 3  
> log(x=8,base=2)  
[1] 3  
> log(base=2,x=8)  
[1] 3  
> log(2,8)  
[1] 0.3333333  
> |
```

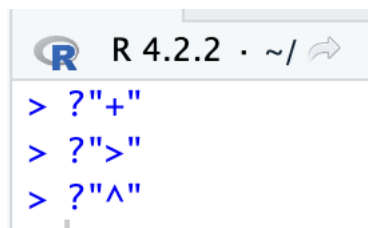
Either you stick to the sequence of the arguments, or you spell out the argument you meant to set

In R, '=' means give, '<-' means assign. Use them at correct occasions



```
R 4.2.2 · ~/   
> x=8  
> log(x,base=2)  
[1] 3  
> log(x=16,base=2)  
[1] 4  
> x  
[1] 8  
> log(x<-16,base=2)  
[1] 4  
> x  
[1] 16
```


Functions



R 4.2.2 · ~/

```
> ?"+"  
> ?">"  
> ?"^"
```

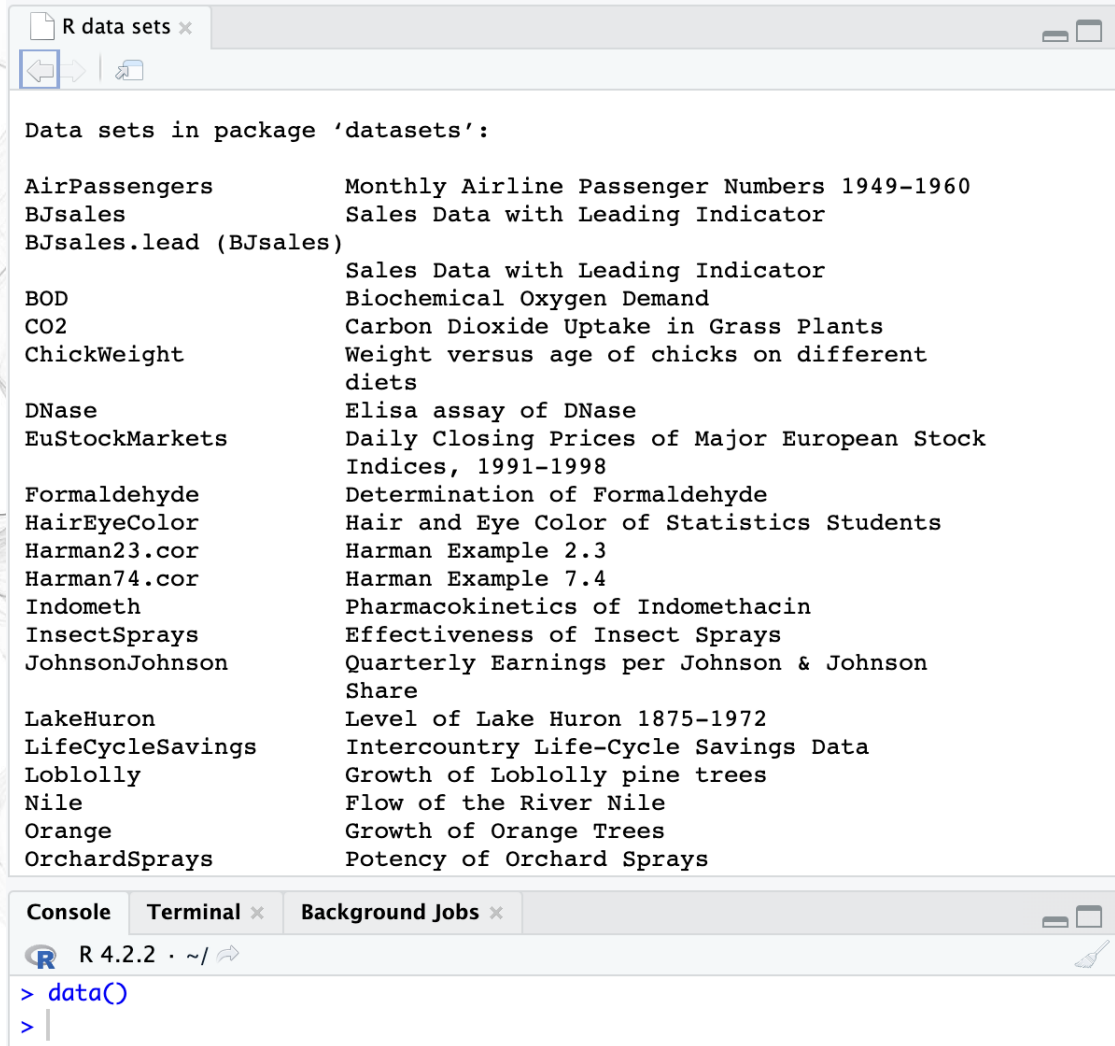
Find helps on arithmetic and relational operators.

Give it a try without parentheses
Ctrl+c if you got stuck into something je ne sais quoi

```
> a<-1  
> rm(a)  
> a  
Error: object 'a' not found  
>
```

Use rm to remove an object

Built-in datasets



The screenshot shows an R console window with the title 'R data sets'. The main pane displays a list of data sets in the 'datasets' package, organized into two columns. The first column lists the dataset names, and the second column provides a brief description for each. The datasets include AirPassengers, BJsales, BJsales.lead, BOD, CO2, ChickWeight, DNase, EuStockMarkets, Formaldehyde, HairEyeColor, Harman23.cor, Harman74.cor, Indometh, InsectSprays, JohnsonJohnson, LakeHuron, LifeCycleSavings, Loblolly, Nile, Orange, and OrchardSprays. At the bottom, the console shows the command `> data()` being entered.

```
R data sets x
Data sets in package 'datasets':

AirPassengers      Monthly Airline Passenger Numbers 1949-1960
BJsales            Sales Data with Leading Indicator
BJsales.lead (BJsales) Sales Data with Leading Indicator
BOD                Biochemical Oxygen Demand
CO2                Carbon Dioxide Uptake in Grass Plants
ChickWeight        Weight versus age of chicks on different
                   diets
DNase              Elisa assay of DNase
EuStockMarkets     Daily Closing Prices of Major European Stock
                   Indices, 1991-1998
Formaldehyde       Determination of Formaldehyde
HairEyeColor       Hair and Eye Color of Statistics Students
Harman23.cor       Harman Example 2.3
Harman74.cor       Harman Example 7.4
Indometh           Pharmacokinetics of Indomethacin
InsectSprays       Effectiveness of Insect Sprays
JohnsonJohnson    Quarterly Earnings per Johnson & Johnson
                   Share
LakeHuron          Level of Lake Huron 1875-1972
LifeCycleSavings   Intercountry Life-Cycle Savings Data
Loblolly           Growth of Loblolly pine trees
Nile               Flow of the River Nile
Orange             Growth of Orange Trees
OrchardSprays      Potency of Orchard Sprays

Console  Terminal x  Background Jobs x
R 4.2.2 · ~/
> data()
>
```

PlantGrowth
Puromycin
Seatbelts
Theoph
Titanic
ToothGrowth

UCBAdmissions
UKDriverDeaths
UKgas
USAccDeaths
USArrests
USJudgeRatings

USPersonalExpenditure

UScitiesD

VADeaths
WWWusage
WorldPhones
ability.cov
airmiles

airquality
anscombe

attenu
attitude
austres

beaver1 (beavers)
beaver2 (beavers)
cars

Results from an Experiment on Plant Growth
Reaction Velocity of an Enzymatic Reaction
Road Casualties in Great Britain 1969-84
Pharmacokinetics of Theophylline
Survival of passengers on the Titanic
The Effect of Vitamin C on Tooth Growth in
Guinea Pigs
Student Admissions at UC Berkeley
Road Casualties in Great Britain 1969-84
UK Quarterly Gas Consumption
Accidental Deaths in the US 1973-1978
Violent Crime Rates by US State
Lawyers' Ratings of State Judges in the US
Superior Court

Personal Expenditure Data
Distances Between European Cities and
Between US Cities
Death Rates in Virginia (1940)
Internet Usage per Minute
The World's Telephones
Ability and Intelligence Tests
Passenger Miles on Commercial US Airlines,
1937-1960
New York Air Quality Measurements
Anscombe's Quartet of 'Identical' Simple
Linear Regressions
The Joyner-Boore Attenuation Data
The Chatterjee-Price Attitude Data
Quarterly Time Series of the Number of
Australian Residents
Body Temperature Series of Two Beavers
Body Temperature Series of Two Beavers
Speed and Stopping Distances of Cars

Many more: `data(package = .packages(all.available = TRUE))`

Built-in datasets

```
> pi  
[1] 3.141593
```

```
> co2
```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1959	315.42	316.31	316.50	317.56	318.13	318.00	316.39	314.65	313.68	313.18	314.66	315.43
1960	316.27	316.81	317.42	318.87	319.87	319.43	318.01	315.74	314.00	313.68	314.84	316.03
1961	316.73	317.54	318.38	319.31	320.42	319.61	318.42	316.63	314.83	315.16	315.94	316.85
1962	317.78	318.40	319.53	320.42	320.85	320.45	319.45	317.25	316.11	315.27	316.53	317.53
1963	318.58	318.92	319.70	321.22	322.08	321.31	319.58	317.61	316.05	315.83	316.91	318.20
1964	319.41	320.07	320.74	321.40	322.06	321.73	320.27	318.54	316.54	316.71	317.53	318.55
1965	319.27	320.28	320.73	321.97	322.00	321.71	321.05	318.71	317.66	317.14	318.70	319.25
1966	320.46	321.43	322.23	323.54	323.91	323.59	322.24	320.20	318.48	317.94	319.63	320.87
1967	322.17	322.34	322.88	324.25	324.83	323.93	322.38	320.76	319.10	319.24	320.56	321.80
1968	322.40	322.99	323.73	324.86	325.40	325.20	323.98	321.95	320.18	320.09	321.16	322.74
1969	323.83	324.26	325.47	326.50	327.21	326.54	325.72	323.50	322.22	321.62	322.69	323.95
1970	324.89	325.82	326.77	327.97	327.91	327.50	326.18	324.53	322.93	322.90	323.85	324.96
1971	326.01	326.51	327.01	327.62	328.76	328.40	327.20	325.27	323.20	323.40	324.63	325.85
1972	326.60	327.47	327.58	329.56	329.90	328.92	327.88	326.16	324.68	325.04	326.34	327.39
1973	328.37	329.40	330.14	331.33	332.31	331.90	330.70	329.15	327.35	327.02	327.99	328.48
1974	329.18	330.55	331.32	332.48	332.92	332.08	331.01	329.23	327.27	327.21	328.29	329.41
1975	330.23	331.25	331.87	333.14	333.80	333.43	331.73	329.90	328.40	328.17	329.32	330.59
1976	331.58	332.39	333.33	334.41	334.71	334.17	332.89	330.77	329.14	328.78	330.14	331.52
1977	332.75	333.24	334.53	335.90	336.57	336.10	334.76	332.59	331.42	330.98	332.24	333.68
1978	334.80	335.22	336.47	337.59	337.84	337.72	336.37	334.51	332.60	332.38	333.75	334.78
1979	336.05	336.59	337.79	338.71	339.30	339.12	337.56	335.92	333.75	333.70	335.12	336.56
1980	337.84	338.19	339.91	340.60	341.2							8.04
1981	339.06	340.30	341.21	342.33	342.7							9.44
1982	340.57	341.44	342.53	343.39	343.9							0.32
1983	341.20	342.35	342.93	344.77	345.5							2.82
1984	343.52	344.33	345.11	346.88	347.2							4.04
1985	344.79	345.82	347.25	348.17	348.7							5.38
1986	346.11	346.78	347.68	349.37	350.0							6.72
1987	347.84	348.29	349.23	350.80	351.6							8.78
1988	350.25	351.54	352.05	353.41	354.0							1.18
1989	352.60	352.92	353.53	355.26	355.5							2.37
1990	353.50	354.55	355.23	356.04	357.0							4.07
1991	354.59	355.63	357.03	358.48	359.2							4.89



```
> library(dslabs)
```

```
> olive
```

	region	area	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic
1	Southern Italy North-Apulia	10.75	0.75	2.26	78.23	6.72	0.36	
2	Southern Italy North-Apulia	10.88	0.73	2.24	77.09	7.81	0.31	
3	Southern Italy North-Apulia	9.11	0.54	2.46	81.13	5.49	0.31	
4	Southern Italy North-Apulia	9.66	0.57	2.40	79.52	6.19	0.50	
5	Southern Italy North-Apulia	10.51	0.67	2.59	77.71	6.72	0.50	
6	Southern Italy North-Apulia	9.11	0.49	2.68	79.24	6.78	0.51	
7	Southern Italy North-Apulia	9.22	0.66	2.64	79.90	6.18	0.49	
8	Southern Italy North-Apulia	11.00	0.61	2.35	77.28	7.34	0.39	
9	Southern Italy North-Apulia	10.82	0.60	2.39	77.45	7.09	0.46	
10	Southern Italy North-Apulia	10.37	0.55	2.13	79.44	6.33	0.26	
11	Southern Italy North-Apulia	10.51	0.35	2.19	79.78	6.05	0.21	
12	Southern Italy North-Apulia	10.36	0.59	2.35	78.68	6.61	0.30	
13	Southern Italy North-Apulia	10.74	0.70	2.14	77.28	7.47	0.50	
14	Southern Italy North-Apulia	8.75	0.52	2.43	80.18	6.55	0.41	
15	Southern Italy North-Apulia	9.52	0.49	2.54	77.95	7.80	0.50	
16	Southern Italy North-Apulia	11.55	0.98	2.01	76.06	8.16	0.32	
17	Southern Italy North-Apulia	9.43	0.94	1.83	78.40	7.88	0.42	
18	Southern Italy North-Apulia	12.78	0.69	2.05	73.44	9.57	0.45	

```
> library(tidyverse)
```

```
> fruit
```

[1] "apple"	"apricot"	"avocado"	"banana"
[5] "bell pepper"	"bilberry"	"blackberry"	"blackcurrant"
[9] "blood orange"	"blueberry"	"boysenberry"	"breadfruit"
[13] "canary melon"	"cantaloupe"	"cherimoya"	"cherry"
[17] "chili pepper"	"clementine"	"cloudberry"	"coconut"
[21] "cranberry"	"cucumber"	"currant"	"damson"
[25] "date"	"dragonfruit"	"durian"	"eggplant"
[29] "elderberry"	"feijoa"	"fig"	"goji berry"
[33] "gooseberry"	"grape"	"grapefruit"	"guava"
[37] "honeydew"	"huckleberry"	"jackfruit"	"jambul"



Atomic data types

```
> a <- 3
> class(a)
[1] "numeric"
> b <- 3:3
> class(b)
[1] "integer"
> c <- "3"
> class(c)
[1] "character"
```

- Variables in R can be of different types. Use the function `class` for help.
- (Major) atomic data types in R:
 - `numeric` (float/double)
 - `integer` (byte, short, int, long, etc.)
 - `logical` (bool)
 - `character` (string)
 - `factor`, will be covered later



Atomic data types

What's the difference between a numeric and an integer in R?

```
> class(1)
[1] "numeric"
> class(1L)
[1] "integer"
> class(as.integer(1))
[1] "integer"
```

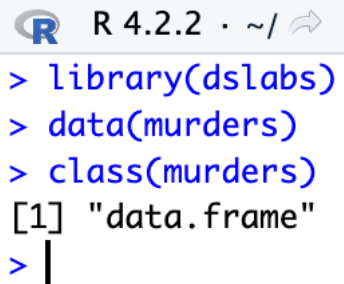
```
> is.integer(1)
[1] FALSE
> is.integer(1L)
[1] TRUE
> is.numeric(1)
[1] TRUE
> is.numeric(1L)
[1] TRUE
> is.double(1)
[1] TRUE
> is.double(1L)
[1] FALSE
```

- In most cases they are interchangeable, automatically coerced
- `integer` is mostly used for indexing

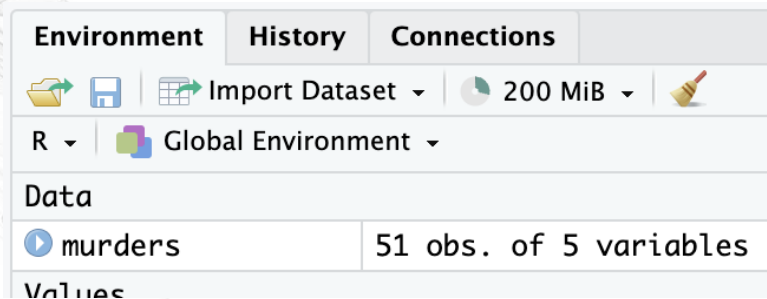
```
> c(1, 2, 3, 4)[1.99] == c(1, 2, 3, 4)[1]
[1] TRUE
```

<https://stackoverflow.com/questions/23660094/whats-the-difference-between-integer-class-and-numeric-class-in-r>

A complex data type - data frame

A screenshot of the R console showing the following commands and output:

```
R 4.2.2 · ~/   
> library(dslabs)  
> data(murders)  
> class(murders)  
[1] "data.frame"  
> |
```

A screenshot of the R IDE's Environment pane. It shows the 'murders' dataset loaded in the Global Environment. The table below represents the data shown in the screenshot.

Environment	History	Connections
R 4.2.2 · ~/		
Import Dataset 200 MiB		
R Global Environment		
Data		
murders	51 obs. of 5 variables	
Values		

Up to now, the variables we have defined are just one number. This is not very useful for storing data. The most common way of storing a dataset in R is in a *data frame*

Conceptually, we can think of a data frame as a table with rows representing observations and the different variables reported for each observation defining the columns. Data frames are particularly useful for datasets because we can combine different data types into one object

A large proportion of data analysis challenges start with data stored in a data frame. Using dataset “murders” as an example, you can access this dataset by loading the dslabs library and loading the murders dataset using the data function

The "murders" dataset

> ?murders

US gun murders by state for 2010

Description

Gun murder data from FBI reports. Also contains the population of each state.

Usage

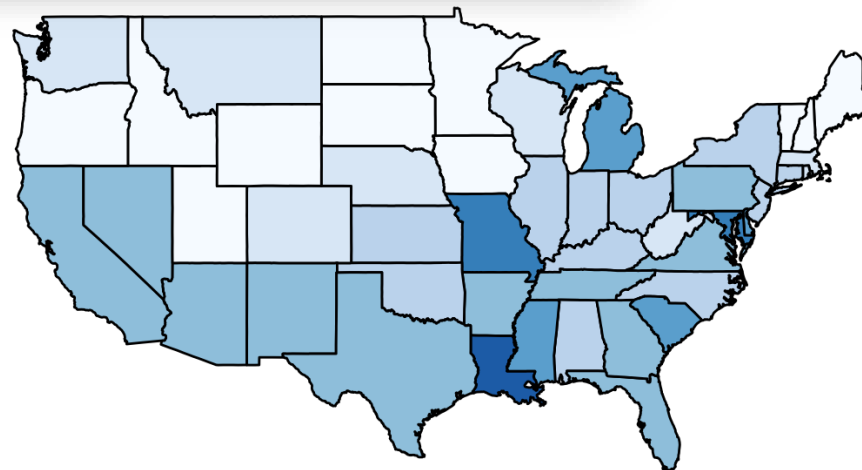
`murders`

Format

An object of class "data.frame".

Details

- `state`. US state
- `abb`. Abbreviation of US state
- `region`. Geographical US region
- `population`. State population (2010)
- `total`. Number of gun murders in state (2010)



> murders

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65
7	Connecticut	CT	Northeast	3574097	97
8	Delaware	DE	South	897934	38
9	District of Columbia	DC	South	601723	99
10	Florida	FL	South	19687653	669
11	Georgia	GA	South	9920000	376
12	Hawaii	HI	West	1360301	7
13	Idaho	ID	West	1567582	12
14	Illinois	IL	North Central	12830632	364
15	Indiana	IN	North Central	6483802	142
16	Iowa	IA	North Central	3046355	21
17	Kansas	KS	North Central	2853118	63
18	Kentucky	KY	South	4339367	116
19	Louisiana	LA	South	4533372	351
20	Maine	ME	Northeast	1328361	11
21	Maryland	MD	South	5773552	293
22	Massachusetts	MA	Northeast	6547629	118
23	Michigan	MI	North Central	9883640	413
24	Minnesota	MN	North Central	5303925	53
25	Mississippi	MS	South	2967297	120
26	Missouri	MO	North Central	5988927	321
27	Montana	MT	West	989415	12
28	Nebraska	NE	North Central	1826341	32
29	Nevada	NV	West	2700551	84
30	New Hampshire	NH	Northeast	1316470	5
31	New Jersey	NJ	Northeast	8791894	246
32	New Mexico	NM	West	2059179	67
33	New York	NY	Northeast	19378102	517
34	North Carolina	NC	South	9535483	286
35	North Dakota	ND	North Central	672591	4
36	Ohio	OH	North Central	11536504	310
37	Oklahoma	OK	South	3751351	111
38	Oregon	OR	West	3831074	36
39	Pennsylvania	PA	Northeast	12702379	457
40	Rhode Island	RI	Northeast	1052567	16
41	South Carolina	SC	South	4625364	207
42	South Dakota	SD	North Central	814180	8
43	Tennessee	TN	South	6346105	219
44	Texas	TX	South	25145561	805
45	Utah	UT	West	2763885	22
46	Vermont	VT	Northeast	625741	2
47	Virginia	VA	South	8001024	250
48	Washington	WA	West	6724540	93
49	West Virginia	WV	South	1852994	27
50	Wisconsin	WI	North Central	5686986	97
51	Wyoming	WY	West	563626	5



Examining an object

The function `str` is useful for finding out more about the structure of an object

```
> str(murders)
```

```
'data.frame':   51 obs. of  5 variables:
 $ state      : chr  "Alabama" "Alaska" "Arizona" "Arkansas" ...
 $ abb       : chr  "AL" "AK" "AZ" "AR" ...
 $ region    : Factor w/ 4 levels "Northeast","South",...: 2 4 4 2 4 4 1 2 2 2 ...
 $ population: num  4779736 710231 6392017 2915918 37253956 ...
 $ total     : num  135 19 232 93 1257 ...
```

```
> head(murders)
```

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65

Print the first six lines instead of print all lines.

What if I want to print 10 lines?

The accessor: \$

For our analysis, we will need to access the different variables represented by columns included in this data frame. To do this, we use the accessor operator \$ in the following way

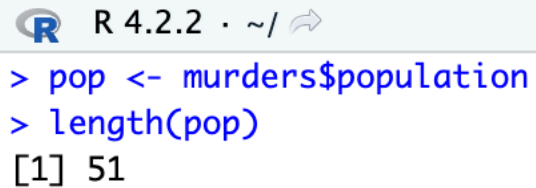
```
R 4.2.2 · ~/ ↗  
> murders$population  
[1] 4779736 710231 6392017 2915918 37253956 5029196 3574097 897934 601723  
[10] 19687653 9920000 1360301 1567582 12830632 6483802 3046355 2853118 4339367  
[19] 4533372 1328361 5773552 6547629 9883640 5303925 2967297 5988927 989415  
[28] 1826341 2700551 1316470 8791894 2059179 19378102 9535483 672591 11536504  
[37] 3751351 3831074 12702379 1052567 4625364 814180 6346105 25145561 2763885  
[46] 625741 8001024 6724540 1852994 5686986 563626  
>
```

But how did we know to use population? Previously, by applying the function str to the object murders, we revealed the names for each of the five variables stored in this table. We can quickly access the variable names using names

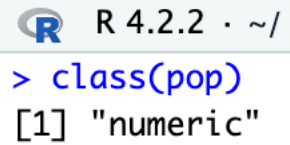
```
R 4.2.2 · ~/ ↗  
> names(murders)  
[1] "state" "abb" "region" "population" "total"
```

It is important to know that the order of the entries in murders\$population **preserves the order of the rows** in our data table. This will later permit us to manipulate one variable based on the results of another

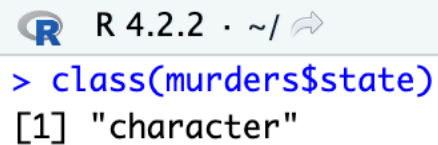
Vector of numeric / character

A screenshot of the R console showing the assignment of the population vector and its length.

```
R 4.2.2 · ~/   
> pop <- murders$population  
> length(pop)  
[1] 51
```

A screenshot of the R console showing the class of the population vector.

```
R 4.2.2 · ~/   
> class(pop)  
[1] "numeric"
```

A screenshot of the R console showing the class of the state names vector.

```
R 4.2.2 · ~/   
> class(murders$state)  
[1] "character"
```

The object `murders$population` is not one number but several. We call these types of objects vectors. A single number is technically a vector of length 1, but in general we use the term vectors to refer to objects with several entries. The function `length` tells you how many entries are in the vector

This particular vector is numeric since population sizes are numbers. In a numeric vector, every entry must be a number

To store character strings, vectors can also be of class character. For example, the state names are characters. As with numeric vectors, all entries in a character vector need to be a character

Vector of logical

```
R 4.2.2 · ~/ ↵  
> z <- 3 == 2  
> z  
[1] FALSE  
> class(z)  
[1] "logical"
```

Another important type of vector is logical vector. These must be either TRUE or FALSE. Here the == is a relational operator asking if 3 is equal to 2. In R, if you just use one =, you actually assign a variable, but if you use two == you test for equality

```
R 4.2.2 · ~/ ↵  
> ?Comparison  
> |
```

Also check other relational operators using ?Comparison

What's the difference between a numeric and an integer in R?

```
R 4.2.2 · ~/ ↵  
> class(1)  
[1] "numeric"  
> class(1L)  
[1] "integer"  
> class(as.integer(1))  
[1] "integer"
```

```
R 4.2.2 · ~/ ↵  
> is.integer(1)  
[1] FALSE  
> is.integer(1L)  
[1] TRUE  
> is.numeric(1)  
[1] TRUE  
> is.double(1)  
[1] TRUE
```

<https://stackoverflow.com/questions/23660094/whats-the-difference-between-integer-class-and-numeric-class-in-r>

Factor

```
R 4.2.2 · ~/ 
> class(murders$region)
[1] "factor"
```

In the murders dataset, we might expect the region to also be a character vector. However, it is not

It is a factor. Factors are useful for storing categorical data. We can see that there are only 4 regions by using the levels function

```
R 4.2.2 · ~/ 
> levels(murders$region)
[1] "Northeast" "South" "North Central" "West"
```

In the background, R stores these levels as integers and keeps a map to keep track of the labels. This is more memory efficient than storing all the characters

> murders					
	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65
7	Connecticut	CT	Northeast	3574097	97
8	Delaware	DE	South	897934	38
9	District of Columbia	DC	South	601723	99
10	Florida	FL	South	19687653	669
11	Georgia	GA	South	9920000	376
12	Hawaii	HI	West	1360301	7
13	Idaho	ID	West	1567582	12
14	Illinois	IL	North Central	12830632	364
15	Indiana	IN	North Central	6483802	142
16	Iowa	IA	North Central	3046355	21
17	Kansas	KS	North Central	2853118	63
18	Kentucky	KY	South	4339367	116
19	Louisiana	LA	South	4533372	351
20	Maine	ME	Northeast	1328361	11
21	Maryland	MD	South	5773552	293
22	Massachusetts	MA	Northeast	6547629	118
23	Michigan	MI	North Central	9883640	413
24	Minnesota	MN	North Central	5303925	53
25	Mississippi	MS	South	2967297	120
26	Missouri	MO	North Central	5988927	321
27	Montana	MT	West	989415	12
28	Nebraska	NE	North Central	1826341	32
29	Nevada	NV	West	2700551	84
30	New Hampshire	NH	Northeast	1316470	5
31	New Jersey	NJ	Northeast	8791894	246
32	New Mexico	NM	West	2059179	67
33	New York	NY	Northeast	19378102	517
34	North Carolina	NC	South	9535483	286
35	North Dakota	ND	North Central	672591	4
36	Ohio	OH	North Central	11536504	310
37	Oklahoma	OK	South	3751351	111
38	Oregon	OR	West	3831074	36
39	Pennsylvania	PA	Northeast	12702379	457
40	Rhode Island	RI	Northeast	1052567	16
41	South Carolina	SC	South	4625364	207
42	South Dakota	SD	North Central	814180	8
43	Tennessee	TN	South	6346105	219
44	Texas	TX	South	25145561	805
45	Utah	UT	West	2763885	22
46	Vermont	VT	Northeast	625741	2
47	Virginia	VA	South	8001024	250
48	Washington	WA	West	6724540	93
49	West Virginia	WV	South	1852994	27
50	Wisconsin	WI	North Central	5686986	97
51	Wyoming	WY	West	563626	5



Factor

```
> factor(x="C", levels=c("C", "A", "B"))  
[1] C  
Levels: C A B
```

Note that the levels have an order that is different from the order of appearance in the factor object. Often we want the levels to follow a specific order. You can specify an order through the levels argument when creating the factor with the factor function. For example, in the murders dataset regions are ordered from east to west. The function reorder lets us change the order of the levels of a factor variable based on a summary computed on a numeric vector. We will demonstrate this with a simple example, and will see more advanced ones in the Data Visualization part

```
R 4.2.2 · ~/ ➡  
> region <- murders$region  
> value <- murders$total  
> region <- reorder(region, value, FUN = sum)  
> levels(region)  
[1] "Northeast"      "North Central" "West"          "South"
```

The new order is in agreement with the fact that the Northeast has the least murders and the South has the most

List

Data frames are a special case of lists. Lists are useful because you can store any combination of different types. You can create a list using the list function like this

```
R 4.2.2 · ~/ 
> record <- list(name = "Tai Man Chan",
+               student_id = 1234,
+               grades = c(95, 82, 91, 97, 93),
+               final_grade = "A")
> |
```

```
> grades <- c(95, 82, 91, 97, 93)
> class(grades)
[1] "numeric"
> grades <- c("95", "82", "91", "97", "93")
> class(grades)
[1] "character"
> grades <- c(95, 82, "91", "97", "93")
> class(grades)
[1] "character"
> class(grades[1])
[1] "character"
> grades[1]
[1] "95"
```

Function c creates a vector, which stands for concatenate. A vector can be a part of a list. List stores any combination of different types, but vector cannot. The example on the left shows that, while numeric can be converted to characters, but not the reverse

When given different types to function c, lower-level data types (e.g., numeric) will be converted (or coerced in R terms) to high-level data types (e.g., characters)

This is a main source of tricky bugs in R, so always check the data types

```
R 4.2.2 · ~/ 
> record
$name
[1] "Tai Man Chan"

$student_id
[1] 1234

$grades
[1] 95 82 91 97 93

$final_grade
[1] "A"
```

List

```
R R 4.2.2 · ~/ ↗  
> record$student_id  
[1] 1234
```

```
R R 4.2.2 · ~/ ↗  
> record[["student_id"]]  
[1] 1234
```

```
R R 4.2.2 · ~/ ↗  
> record[["student_id"]]  
[1] 1234  
> record["student_id"]  
$student_id  
[1] 1234  
  
> class(record[["student_id"]])  
[1] "numeric"  
> class(record["student_id"])  
[1] "list"
```

As like data frames, you can extract the components of a list with the accessor `$`

We can also use double square brackets (`[[ ]`) like this

Double square brackets return the value. Single square bracket returns a list with the key and the value

List

```
R 4.2.2 · ~/ ↵  
> record2 <- list("Tai Man Chan", 1234)  
> record2  
[[1]]  
[1] "Tai Man Chan"  
  
[[2]]  
[1] 1234
```

```
R 4.2.2 · ~/ ↵  
> record2[[1]]  
[1] "Tai Man Chan"  
> record2[1]  
[[1]]  
[1] "Tai Man Chan"
```

You might also see a list without key but just values

If a list does not have names, you cannot extract the elements with \$, but you can still use the brackets method and instead of providing the variable name, you provide the list index, like this

Feature	[]	[[]]
Output Type	Same as input (e.g., list, vector)	Raw content (e.g., number, character)
Number of Elements	Can return multiple elements	Returns only one element
Hierarchy	Preserves structure	Drops one level (e.g., list → element)
Name-Based Subsetting	Works (<code>list["a"]</code>)	Works (<code>list[["a"]]</code> or <code>list\$a</code>)

List

OK, before we start with the matrix data type, let us get it straight, the index in R starts with 1

```
R 4.2.2 · ~/   
> record2[[1]]  
[1] "Tai Man Chan"  
> record2[[0]]  
Error in record2[[0]] :  
  attempt to select less than one element in get1index <real>
```



Index with 0

```
R 4.2.2 · ~/ ↵  
> v<-c("a","b","c")  
> v[1]  
[1] "a"  
> v[0]  
character(0)  
> class(v[1])  
[1] "character"  
> class(v[0])  
[1] "character"
```

```
> v<-c(1,2,3)  
> v[1]  
[1] 1  
> v[0]  
numeric(0)  
> class(v[1])  
[1] "numeric"  
> class(v[0])  
[1] "numeric"
```

```
R 4.2.2 · ~/ ↵  
> l<-list(1,2,3)  
> l[[1]]  
[1] 1  
> l[1]  
[[1]]  
[1] 1  
  
> l[[0]]  
Error in l[[0]] :  
  attempt to select less than one element in get1index <real>  
> l[0]  
list()  
> |
```

Visiting index 0 gives you the data type of the object

But with data frame that we will introduce later, we would seldom use list



Index with -1

```
> x <- c(1, 2, 3, 4, 5)
> x[-1]
[1] 2 3 4 5
> x[-2]
[1] 1 3 4 5
```

Indexing with negative number
(`[-1]`, not `[[-1]]`) gives the
same data structure, but with
the specified item excluded

```
> x <- list(item1 = 1, item2 = 2, item3 = 3)
> x[-1]
$item2
[1] 2

$item3
[1] 3

> x[-2]
$item1
[1] 1

$item3
[1] 3
```

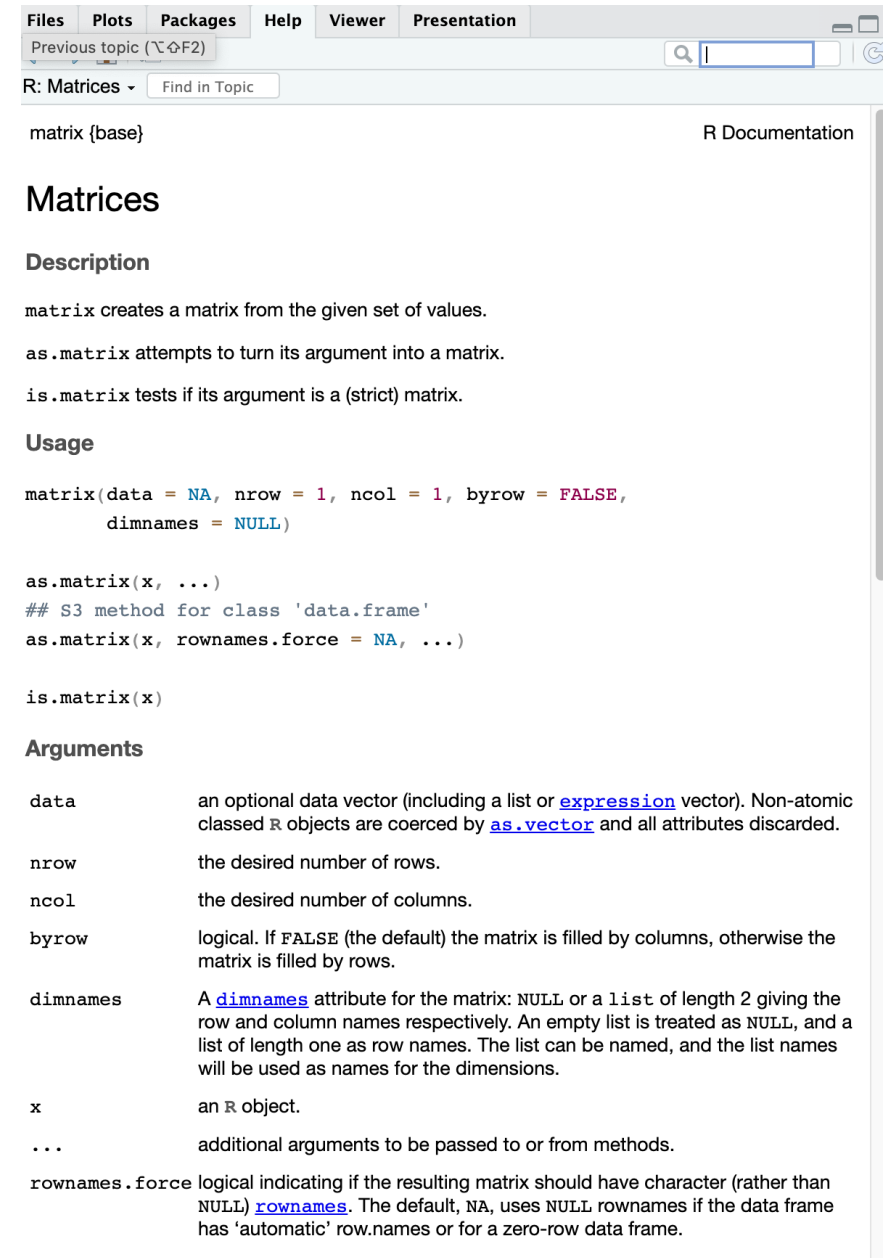
See also: <https://stackoverflow.com/questions/7336679/in-r-what-does-a-negative-index-do>

Index with -1

Language	Index Base	-1 Meaning	Example	Notes
R	1-based	Exclude element at position 1 (drop by position)	<code>x <- 1:5; x[-1] #</code> <code>2 3 4 5</code>	Cannot mix positive and negative indices. To get last element, use <code>x[length(x)]</code> or <code>tail(x, 1)</code> .
Python (lists, NumPy, pandas <code>.iloc</code>)	0-based	Last element (from end)	<code>a = [10, 20, 30];</code> <code>a[-1] # 30</code>	Negative indices count from end: <code>-1</code> last, <code>-2</code> second last.
MATLAB	1-based	Invalid	<code>A(end)</code> for last	Negative indices not allowed. Use <code>end</code> , <code>end-1</code> for second last.
C++ (std::vector, arrays)	0-based	Invalid / UB	<code>v[-1]</code> → undefined behavior	Must use <code>v.back()</code> or <code>v[v.size()-1]</code> . No built-in negative indexing.
JavaScript	0-based	Not last by default; reads property <code>"-1"</code>	<code>arr.at(-1)</code> → last (ES2022+)	<code>arr[-1]</code> usually undefined. Use <code>.at(-1)</code> or <code>arr[arr.length - 1]</code> .

Matrix

- Matrix is another type of object that is common in R. Matrix is similar to a data frame in that it is are two-dimensional with rows and columns. However, like numeric, character and logical vectors, all entries in matrices have to be of the same type
- For this reason, data frames are much more useful for storing data, since we can have characters, factors, and numbers in them
- Let's focus on how to visit specific columns, rows, or elements in matrix because it's similar to data frame



The screenshot shows the R Documentation page for Matrices. The browser window has tabs for Files, Plots, Packages, Help, Viewer, and Presentation. The address bar shows 'Previous topic (\`⇧F2\`)' and a search bar. The page title is 'R: Matrices' with a 'Find in Topic' button. The main content area is titled 'matrix {base}' and 'R Documentation'. Below this is the section 'Matrices'. Under 'Description', it states: 'matrix creates a matrix from the given set of values.', 'as.matrix attempts to turn its argument into a matrix.', and 'is.matrix tests if its argument is a (strict) matrix.' Under 'Usage', it shows the function signature: 'matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE, dimnames = NULL)' followed by 'as.matrix(x, ...)' and '## S3 method for class 'data.frame'' and 'as.matrix(x, rownames.force = NA, ...)' and 'is.matrix(x)'. Under 'Arguments', it lists: 'data' (an optional data vector (including a list or [expression](#) vector). Non-atomic classed R objects are coerced by [as.vector](#) and all attributes discarded.), 'nrow' (the desired number of rows.), 'ncol' (the desired number of columns.), 'byrow' (logical. If FALSE (the default) the matrix is filled by columns, otherwise the matrix is filled by rows.), 'dimnames' (A [dimnames](#) attribute for the matrix: NULL or a list of length 2 giving the row and column names respectively. An empty list is treated as NULL, and a list of length one as row names. The list can be named, and the list names will be used as names for the dimensions.), 'x' (an R object.), '...' (additional arguments to be passed to or from methods.), and 'rownames.force' (logical indicating if the resulting matrix should have character (rather than NULL) [rownames](#). The default, NA, uses NULL rownames if the data frame has 'automatic' row.names or for a zero-row data frame.).

Matrix

```
> 1:12
[1] 1 2 3 4 5 6 7 8 9 10 11 12
> mat <- matrix(1:12, 4, 3)
> mat
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

R 4.2.2 · ~/ ↗

```
> mat[2, 3]
[1] 10
```

R 4.2.2 · ~/ ↗

```
> mat[2, ]
[1] 2 6 10
```

R 4.2.2 · ~/ ↗

```
> mat[, 3]
[1] 9 10 11 12
```

We can define a matrix using the matrix function. We need to specify the number of rows and columns

You can access specific entries in a matrix using square brackets ([]). If you want the second row, third column

If you want the entire second row, you leave the column spot empty. Notice that this returns a vector, not a matrix

Similarly, if you want the entire third column, you leave the row spot empty. This is also a vector, not a matrix



Matrix

```
R 4.2.2 · ~/
```

```
> mat[, 2:3]
```

	[,1]	[,2]
[1,]	5	9
[2,]	6	10
[3,]	7	11
[4,]	8	12

You can access more than one column or more than one row if you like. This will give you a new matrix

```
> mat[1:2, 2:3]
```

	[,1]	[,2]
[1,]	5	9
[2,]	6	10

You can subset both rows and columns

Matrix

```
R 4.2.2 · ~/ ↗  
> as.data.frame(mat)  
  V1 V2 V3  
1  1  5  9  
2  2  6 10  
3  3  7 11  
4  4  8 12
```

We can convert matrices into data frames using the function `as.data.frame`

```
R 4.2.2 · ~/ ↗  
> data("murders")  
> class(murders)  
[1] "data.frame"  
> murders[25, 1]  
[1] "Mississippi"  
> murders[2:3, ]  
  state abb region population total  
2  Alaska AK   West    710231    19  
3  Arizona AZ   West    6392017   232
```

You can also use single square brackets (`[]`) to access rows and columns of a data frame

Name

```
R 4.2.2 · ~/ ↵  
> codes <- c(italy = 380, canada = 124, egypt = 818)  
> codes  
italy canada egypt  
380 124 818
```

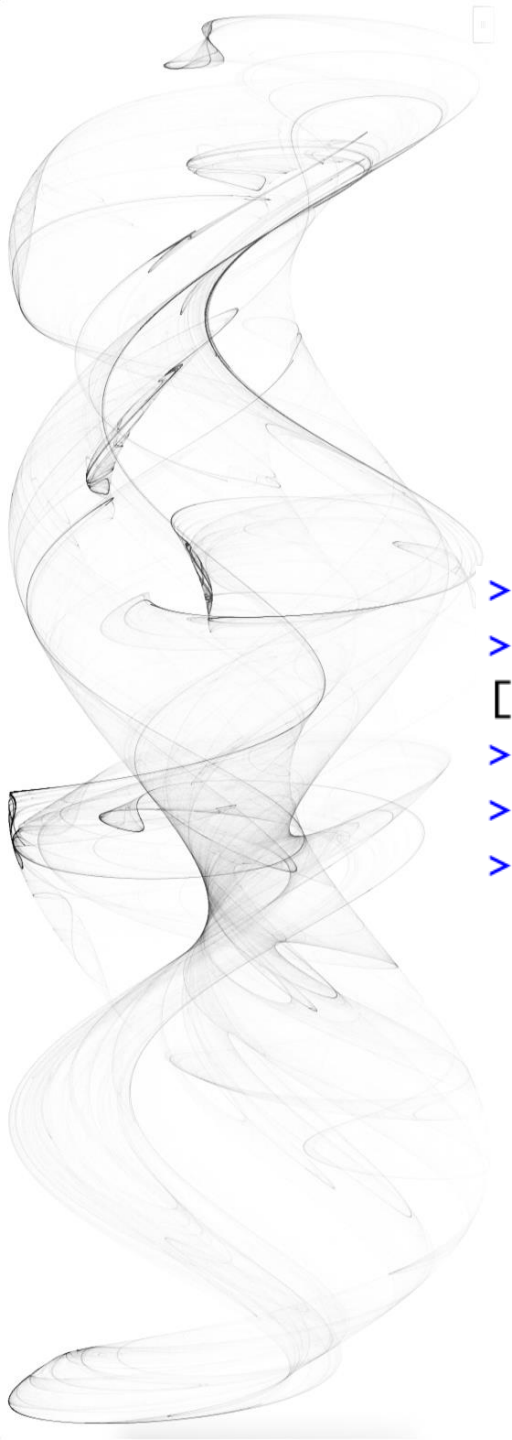
```
R 4.2.2 · ~/ ↵  
> class(codes)  
[1] "numeric"  
> names(codes)  
[1] "italy" "canada" "egypt"
```

```
R 4.2.2 · ~/ ↵  
> codes <- c("italy" = 380, "canada" = 124, "egypt" = 818)  
> codes  
italy canada egypt  
380 124 818
```

Sometimes it is useful to name the entries of a vector. For example, when defining a vector of country codes, we can use the names to connect the two

The object codes continues to be a numeric vector, but with names

If the use of strings without quotes looks confusing, know that you can use the quotes as well



Name

```
> codes <- c(380, 124, 818)
> codes
[1] 380 124 818
> country <- c("italy", "canada", "egypt")
> names(codes) <- country
> codes
italy canada egypt
  380    124    818
```

Names can be given later

Sequence

```
R 4.2.2 · ~/ ↗  
> seq(1,10)  
[1] 1 2 3 4 5 6 7 8 9 10  
> 1:10  
[1] 1 2 3 4 5 6 7 8 9 10
```

Files Plots Packages Help Viewer Presentation

R: Sequence Generation Find in Topic

seq (base) R Documentation

Sequence Generation

Description

Generate regular sequences. `seq` is a standard generic with a default method. `seq.int` is a primitive which can be much faster but has a few restrictions. `seq_along` and `seq_len` are very fast primitives for two common cases.

Usage

```
seq(...)
```

```
## Default S3 method:
```

```
seq(from = 1, to = 1, by = ((to - from)/(length.out - 1)),  
    length.out = NULL, along.with = NULL, ...)
```

Creating a vector with an integer sequence.
The first argument defines the start, and
the second defines the end which is
included

The third argument controls the increment
(aka step)



Sequence

```
R 4.2.2 · ~/ ↗  
> seq(1, 10, 2)  
[1] 1 3 5 7 9  
> seq(1, 10, 3)  
[1] 1 4 7 10  
> seq(1, 10, 4)  
[1] 1 5 9  
> seq(1, 10, 5)  
[1] 1 6  
> seq(1, 10, 6)  
[1] 1 7  
> seq(1, 10, 11)  
[1] 1
```

Examples of how increment argument works

```
R 4.2.2 · ~/ ↗  
> class(1:10)  
[1] "integer"  
> class(seq(1, 10, 0.5))  
[1] "numeric"
```

Beware that data type of a sequence is integer, unless the increment is not

Subsetting

```
R 4.2.2 · ~/ ↵  
> codes[2]  
canada  
124
```

```
R 4.2.2 · ~/ ↵  
> codes[c(1,3)]  
italy egypt  
380 818
```

```
R 4.2.2 · ~/ ↵  
> codes[1:2]  
italy canada  
380 124
```

```
R 4.2.2 · ~/ ↵  
> codes["canada"]  
canada  
124  
> codes[c("egypt", "italy")]  
egypt italy  
818 380
```

We use square brackets to access specific elements of a vector. For the vector `codes` we defined above, we can access the second element like this

You can get more than one entry by using a vector as an index

Integer sequence is particularly useful if we want to access, say, the first two elements

If the elements have names, we can also access the entries using these names

Coercion

- Coercion is an attempt by R to be flexible with data types. When an entry does not match the expected, some of the prebuilt R functions try to guess what was meant before throwing an error. This can also lead to confusion. Failing to understand coercion can drive programmers crazy when attempting to code in R since it behaves quite differently from most other languages in this regard

C/C++ and other
type-safe languages



mcc
@mcclure111

Follow

In C++ we don't say "Missing asterisk" we say "error C2664: 'void std::vector<block,std::allocator<_Ty>>::push_back(const block &)': cannot convert argument 1 from 'std::_Vector_iterator<std::_Vector_val<std::_Simple_types<block>>>' to 'block &&'" and i think that's beautiful

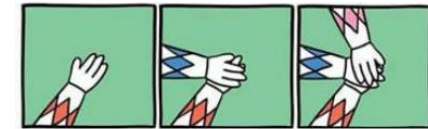
4:30 PM - 1 Jun 2018

292 Retweets 926 Likes



20 292 926

R



1

1, 2

1, 2, 3



1, 2, 3

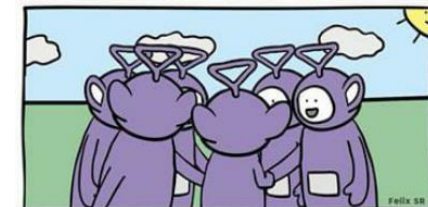
1, 2, 3,

1, 2, 3,

4

4, 5

4, 5, "6"



"1", "2", "3", "4", "5", "6"

Coercion examples

 R 4.2.2 · ~/ 

```
> x <- c(1, "canada", 3)
> x
[1] "1"      "canada" "3"
> class(x)
[1] "character"
```

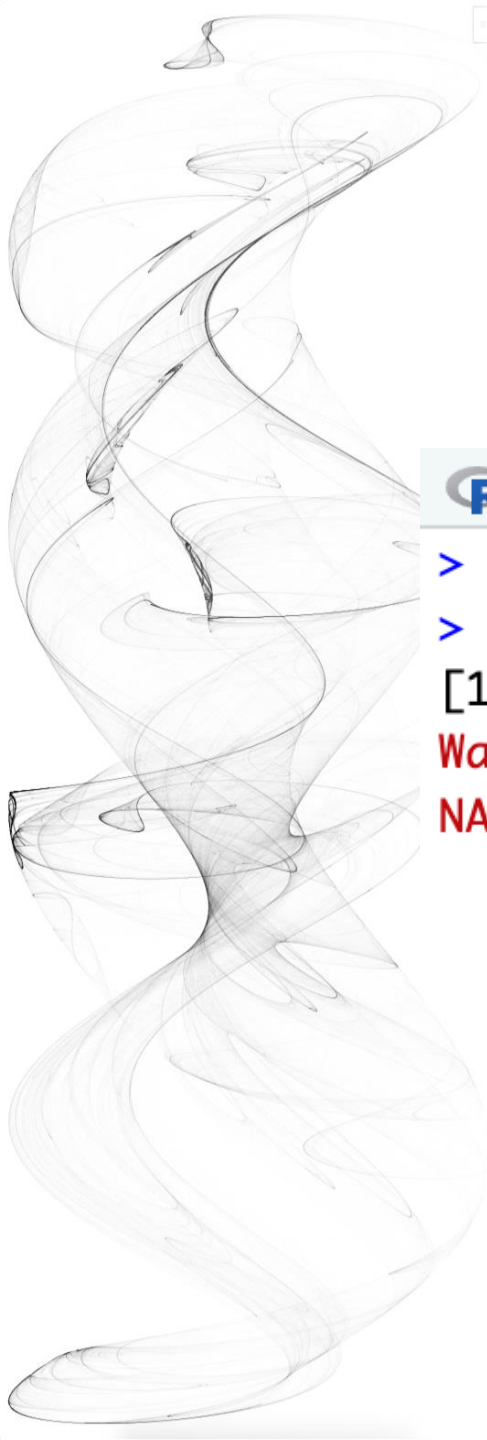
Integer stored as character. R guessed that because you put a character string in the vector, you meant the 1 and 3 to actually be character strings "1" and "3". The fact that not even a warning is issued is an example of how coercion can cause many unnoticed errors in R

 R 4.2.2 · ~/ 

```
> as.numeric("canada")
[1] NA
Warning message:
NAs introduced by coercion
```

And even when a coercion is not possible, it give a warning, not an error, and returns NA (Not Available)

Coercion examples



```
R 4.2.2 · ~/ ↵  
> x <- c("1", "b", "3")  
> as.numeric(x)  
[1] 1 NA 3  
Warning message:  
NAs introduced by coercion
```

R does not have any guesses for what number you want when you type b, so it does not try

As a data scientist you will encounter the NAs often as they are generally used for missing data, a common problem in real-world datasets

Sorting (sort)

Now that we have mastered some basic R knowledge, let's try to gain some insights into the safety of different states in the context of gun murders

R 4.2.2 · ~/

```
> library(dslabs)
> data(murders)
> sort(murders$total)
```

```
[1] 2 4 5 5 7 8 11
[8] 12 12 16 19 21 22 27
[15] 32 36 38 53 63 65 67
[22] 84 93 93 97 97 99 111
[29] 116 118 120 135 142 207 219
[36] 232 246 250 286 293 310 321
[43] 351 364 376 413 457 517 669
[50] 805 1257
```

Say we want to rank the states from least to most gun murders. The function `sort` sorts a vector in increasing order. We can therefore see the largest number of gun murders by this. But how do we know which state had 1257

R 4.2.2 · ~/

> murders

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65
7	Connecticut	CT	Northeast	3574097	97
8	Delaware	DE	South	897934	38
9	District of Columbia	DC	South	601723	99
10	Florida	FL	South	19687653	669
11	Georgia	GA	South	9920000	376
12	Hawaii	HI	West	1360301	7
13	Idaho	ID	West	1567582	12
14	Illinois	IL	North Central	12830632	364
15	Indiana	IN	North Central	6483802	142
16	Iowa	IA	North Central	3046355	21
17	Kansas	KS	North Central	2853118	63
18	Kentucky	KY	South	4339367	116
19	Louisiana	LA	South	4533372	351
20	Maine	ME	Northeast	1328361	11
21	Maryland	MD	South	5773552	293
22	Massachusetts	MA	Northeast	6547629	118
23	Michigan	MI	North Central	9883640	413
24	Minnesota	MN	North Central	5303925	53
25	Mississippi	MS	South	2967297	120
26	Missouri	MO	North Central	5988927	321
27	Montana	MT	West	989415	12
28	Nebraska	NE	North Central	1826341	32
29	Nevada	NV	West	2700551	84
30	New Hampshire	NH	Northeast	1316470	5
31	New Jersey	NJ	Northeast	8791894	246
32	New Mexico	NM	West	2059179	67
33	New York	NY	Northeast	19378102	517
34	North Carolina	NC	South	9535483	286
35	North Dakota	ND	North Central	672591	4
36	Ohio	OH	North Central	11536504	310
37	Oklahoma	OK	South	3751351	111
38	Oregon	OR	West	3831074	36
39	Pennsylvania	PA	Northeast	12702379	457
40	Rhode Island	RI	Northeast	1052567	16
41	South Carolina	SC	South	4625364	207
42	South Dakota	SD	North Central	814180	8
43	Tennessee	TN	South	6346105	219
44	Texas	TX	South	25145561	805
45	Utah	UT	West	2763885	22
46	Vermont	VT	Northeast	625741	2
47	Virginia	VA	South	8001024	250
48	Washington	WA	West	6724540	93
49	West Virginia	WV	South	1852994	27
50	Wisconsin	WI	North Central	5686986	97
51	Wyoming	WY	West	563626	5



Sorting (order)

The function `order` is closer to what we want. It takes a vector as input and returns the vector of indexes that sorts the input vector

```
R 4.2.2 · ~/ 
> x<-c(31, 4, 15, 92, 65)
> sort(x)
[1] 4 15 31 65 92
> index<-order(x)
> index
[1] 2 3 1 5 4
> x[index]
[1] 4 15 31 65 92
```

Rather than sort the input vector, the function `order` returns the index that sorts input vector. If we look at the returned index, we see why it works, the second entry of `x` is the smallest, so `order(x)` starts with 2. The next smallest is the third entry, so the second entry is 3 and so on

Sorting (order)

How does this help us order the states by murders? First, remember that the entries of vectors you access with `$` follow the same order as the rows in the table. For example, the vector that contains state names, is matched by the order of the totals

```
> murders$state[1:6]
[1] "Alabama" "Alaska" "Arizona" "Arkansas"
[5] "California" "Colorado"
```

```
R 4.2.2 · ~/
> index <- order(murders$total)
> murders$state[index]
[1] "Vermont" "North Dakota" "New Hampshire"
[4] "Wyoming" "Hawaii" "South Dakota"
[7] "Maine" "Idaho" "Montana"
[10] "Rhode Island" "Alaska" "Iowa"
[13] "Utah" "West Virginia" "Nebraska"
[16] "Oregon" "Delaware" "Minnesota"
[19] "Kansas" "Colorado" "New Mexico"
[22] "Nevada" "Arkansas" "Washington"
[25] "Connecticut" "Wisconsin" "District of Columbia"
[28] "Oklahoma" "Kentucky" "Massachusetts"
[31] "Mississippi" "Alabama" "Indiana"
[34] "South Carolina" "Tennessee" "Arizona"
[37] "New Jersey" "Virginia" "North Carolina"
[40] "Maryland" "Ohio" "Missouri"
[43] "Louisiana" "Illinois" "Georgia"
[46] "Michigan" "Pennsylvania" "New York"
[49] "Florida" "Texas" "California"
```

It shows Vermont has the least total murders and California had the most total murders



Sorting (max/min and which.max/min)

```
R 4.2.2 · ~/    
> max(murders$total)  
[1] 1257  
> i_max <- which.max(murders$total)  
> murders$state[i_max]  
[1] "California"
```

If we are only interested in the entry with the largest value, we can use `max` for the value, and `which.max` for the index of the largest value

Does this mean California is the most dangerous state? Definitely not. We will work on this later



Sorting (rank)

```
R 4.2.2 · ~/   
> x <- c(31, 4, 15, 92, 65)  
> rank(x)  
[1] 3 1 2 5 4
```

original	sort	order	rank
31	4	2	3
4	15	3	1
15	31	1	2
92	65	5	5
65	92	4	4

Although not as frequently used as `order` and `sort`, the function `rank` is also related to `order` and can be useful. For any given vector it returns a vector with the rank of the first entry, second entry, etc., of the input vector

To summarize, let's look at the results of the three functions we have introduced



Vector arithmetic

California had the most murders, but does this mean it is the most dangerous state? What if it just has many more people than any other state? We can quickly confirm that California indeed has the largest population:

```
R 4.2.2 · ~/ 
> library(dslabs)
> data("murders")
> max(murders$population)
[1] 37253956
> which.max(murders$population)
[1] 5
> murders$state[which.max(murders$population)]
[1] "California"
```

What we really should be computing is the murders per capita. The reports we describe in the motivating section used murders per 100,000 as the unit. To compute this quantity, the powerful vector arithmetic capabilities of R come in handy



Rescaling a vector

In R, arithmetic operations on vectors occur element-wise. For a quick example, suppose we have height in inches, and we want to convert to centimeters. Notice what happens when we multiply inches by 2.54:

```
R 4.2.2 · ~/ ↵  
> inches <- c(69, 62, 66, 70, 70, 73, 67, 73, 67, 70)  
> inches * 2.54  
[1] 175.26 157.48 167.64 177.80 177.80 185.42 170.18 185.42 170.18 177.80
```

The average height of a Hong Kong male at 19-20 is 173.4cm, if for each entry we want to compute how many centimeters taller or shorter than the average we can subtract it from every entry like this:

```
R 4.2.2 · ~/ ↵  
> inches * 2.54 - 173.4  
[1] 1.86 -15.92 -5.76 4.40 4.40 12.02 -3.22 12.02 -3.22 4.40
```

Operate on two vectors

If we have two vectors of the same length, and we sum them in R, they will be added entry by entry as follows. The same holds for other mathematical operations, such as -, * and /

$$\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} + \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} = \begin{pmatrix} a + e \\ b + f \\ c + g \\ d + h \end{pmatrix}$$

Let's compute the murder rates as:

```
R 4.2.2 · ~/
> murder_rate <- murders$total / murders$population * 100000
> murder_rate
 [1] 2.8244238 2.6751860 3.6295273 3.1893901 3.3741383 1.2924531 2.7139722
 [8] 4.2319369 16.4527532 3.3980688 3.7903226 0.5145920 0.7655102 2.8369608
[15] 2.1900730 0.6893484 2.2081106 2.6732010 7.7425810 0.8280881 5.0748655
[22] 1.8021791 4.1786225 0.9992600 4.0440846 5.3598917 1.2128379 1.7521372
[29] 3.1104763 0.3798036 2.7980319 3.2537239 2.6679599 2.9993237 0.5947151
[36] 2.6871225 2.9589340 0.9396843 3.5977513 1.5200933 4.4753235 0.9825837
[43] 3.4509357 3.2013603 0.7959810 0.3196211 3.1246001 1.3829942 1.4571013
[50] 1.7056487 0.8871131
```

Once we do this, we notice that California is no longer near the top of the list:

```
> murders$abb[order(murder_rate)]
 [1] "VT" "NH" "HI" "ND" "IA" "ID" "UT" "ME" "WY" "OR" "SD" "MN" "MT" "CO" "WA" "WV"
[17] "RI" "WI" "NE" "MA" "IN" "KS" "NY" "KY" "AK" "OH" "CT" "NJ" "AL" "IL" "OK" "NC"
[33] "NV" "VA" "AR" "TX" "NM" "CA" "FL" "TN" "PA" "AZ" "GA" "MS" "MI" "DE" "SC" "MD"
[49] "MO" "LA" "DC"
```



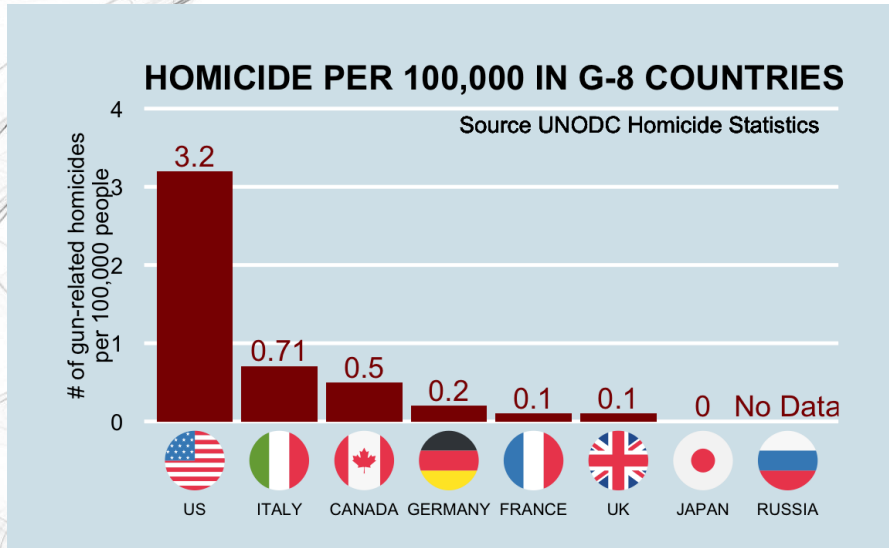

Beware of recycling

Another common source of unnoticed errors in R is the use of recycling. We saw that vectors are added element-wise. So, if the vectors don't match in length, it is natural to assume that we should get an error. But we don't. Notice what happens:

```
R 4.2.2 · ~/ ↵  
> x <- c(1, 2, 3)  
> y <- c(10, 20, 30, 40, 50, 60, 70)  
> x+y  
[1] 11 22 33 41 52 63 71  
Warning message:  
In x + y : longer object length is not a multiple of shorter object length
```

We do get a warning, but no error. For the output, R has recycled the numbers in x. Notice the last digit of numbers in the output

Subsetting with a logical vector



Say, we want to know which states are no worse than Italy.

Note that we get back a logical vector with TRUE for each entry smaller than or equal to 0.71. To see which states these are, we can leverage the fact that vectors can be indexed with a logical vector.

```
R 4.2.2 · ~/
> murder_rate <- murders$total / murders$population * 100000
> ind <- murder_rate <= 0.71
> ind
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
[14] FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[27] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE
[40] FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE
> murders$state[ind]
[1] "Hawaii"      "Iowa"         "New Hampshire" "North Dakota" "Vermont"
> sum(ind)
[1] 5
```

In order to count how many are TRUE, the function `sum` returns the sum of the entries of a vector and logical vectors get coerced to numeric with TRUE coded as 1 and FALSE as 0.

Logical operators

```
R 4.2.2 · ~/ 
> TRUE & TRUE
[1] TRUE
> TRUE && TRUE
[1] TRUE
> TRUE && FALSE
[1] FALSE
> TRUE & FALSE
[1] FALSE
> FALSE & FALSE
[1] FALSE
> ?Logic
```

It's "TRUE", not "true", or "True".

In Python/C, & is "bitwise and", && is "logical and"

In R, both & and && are logical and

```
R 4.2.2 · ~/ 
> vec1 <- c(TRUE, TRUE, FALSE)
> vec2 <- c(TRUE, FALSE, FALSE)
> vec1 & vec2
[1] TRUE FALSE FALSE
> vec1 && vec2
[1] TRUE
Warning messages:
1: In vec1 && vec2 : 'length(x) = 3 > 1' in coercion to 'logical(1)'
2: In vec1 && vec2 : 'length(x) = 3 > 1' in coercion to 'logical(1)'
```

But please always use &
&& assumes length 1 on input and only compares once

<https://stackoverflow.com/questions/6558921/boolean-operators-and>

which & match

R 4.2.2 · ~/

```
> murders$state == "California"
[1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[14] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[27] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[40] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
> which(murders$state == "California")
[1] 5
> murder_rate[which(murders$state == "California")]
[1] 3.374138
```

R 4.2.2 · ~/

```
> murder_rate[which(murders$state == "California" | murders$state == "Florida")]
[1] 3.374138 3.398069
> murder_rate[which(murders$state == "Florida" | murders$state == "California")]
[1] 3.374138 3.398069
> murder_rate[match(c("California", "Florida"), murders$state)]
[1] 3.374138 3.398069
> murder_rate[match(c("Florida", "California"), murders$state)]
[1] 3.398069 3.374138
```

R 4.2.2 · ~/

```
> murder_rate[which(murders$state == "Hong Kong" | murders$state == "California")]
[1] 3.374138
> murder_rate[match(c("Hong Kong", "California"), murders$state)]
[1] NA 3.374138
```

Suppose we want to look up California's murder rate. For this type of operation, it is convenient to convert vectors of logical into indexes. The function which tells us which entries of a logical vector are TRUE

If instead of just one state we want to find out the murder rates for several states, say New York and Florida, we can use the function match. Notice that match output indices according to your input vector

Behavior on mismatch is different between using match and using a logical combination



%in%

If rather than an index we want a logical vector that tells us whether or not each element of a first vector is in a second, we can use the function %in%. Let's imagine you are not sure if Boston, Dakota, and Washington are states. You can find out like this:

```
R 4.2.2 · ~/ ➔
```

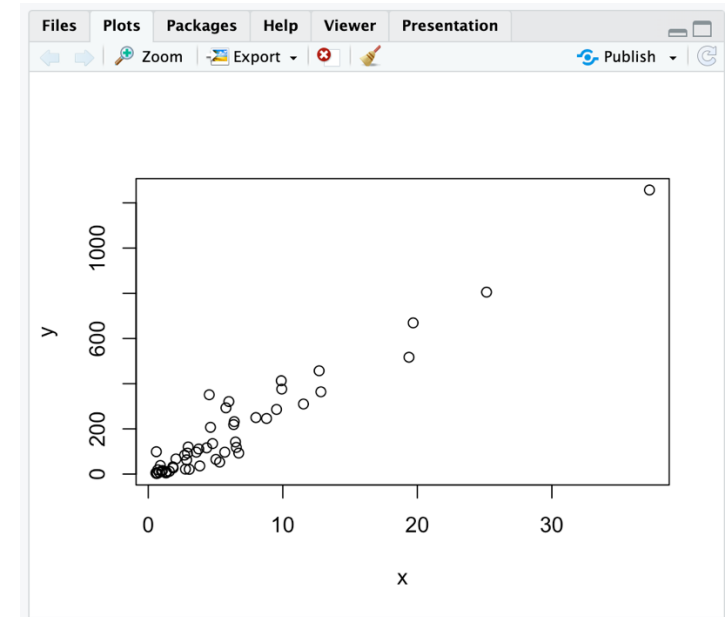
```
> c("Boston", "Dakota", "Washington") %in% murders$state  
[1] FALSE FALSE  TRUE
```


Basic plots

Just first have a feeling of how to generate figures and where to find them. We will learn visualization a lot more in later classes

A scatterplot

```
R 4.2.2 · ~/   
> x <- murders$population / 10^6  
> y <- murders$total  
> plot(x, y)
```



Also try this:

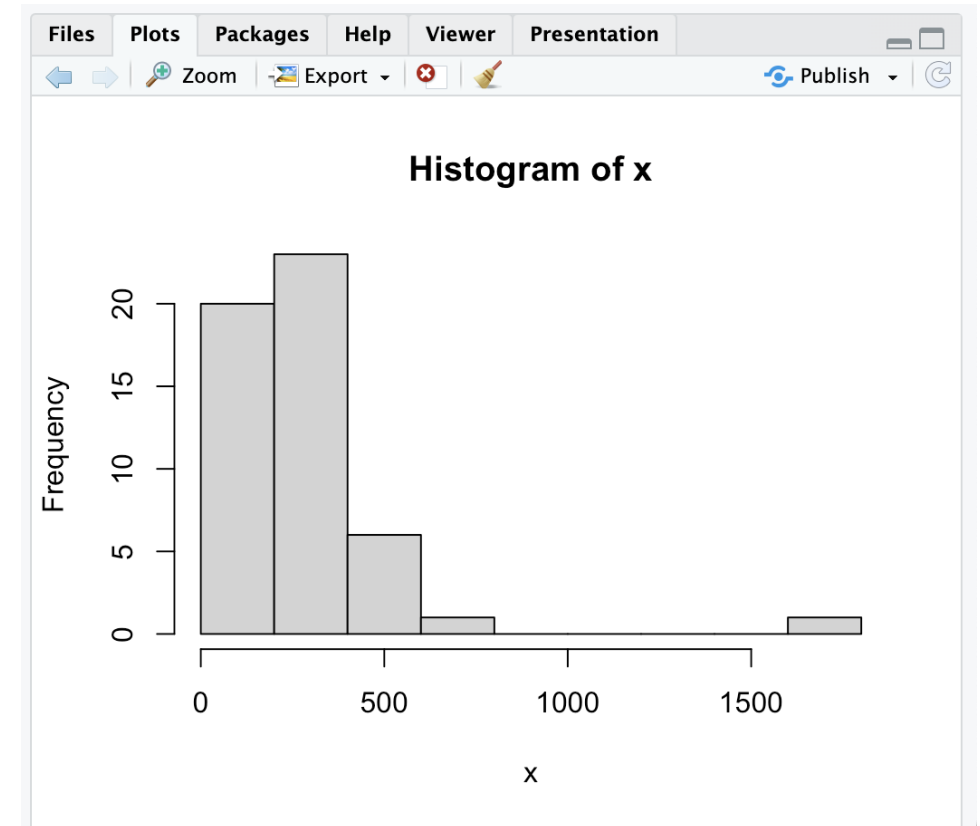
```
R 4.2.2 · ~/   
> with(murders, plot(population / 10^6, total))
```

Histogram for visualizing distribution

Just first have a feeling of how to generate figures and where to find them. We will learn visualization a lot more in later classes

R 4.2.2 · ~/

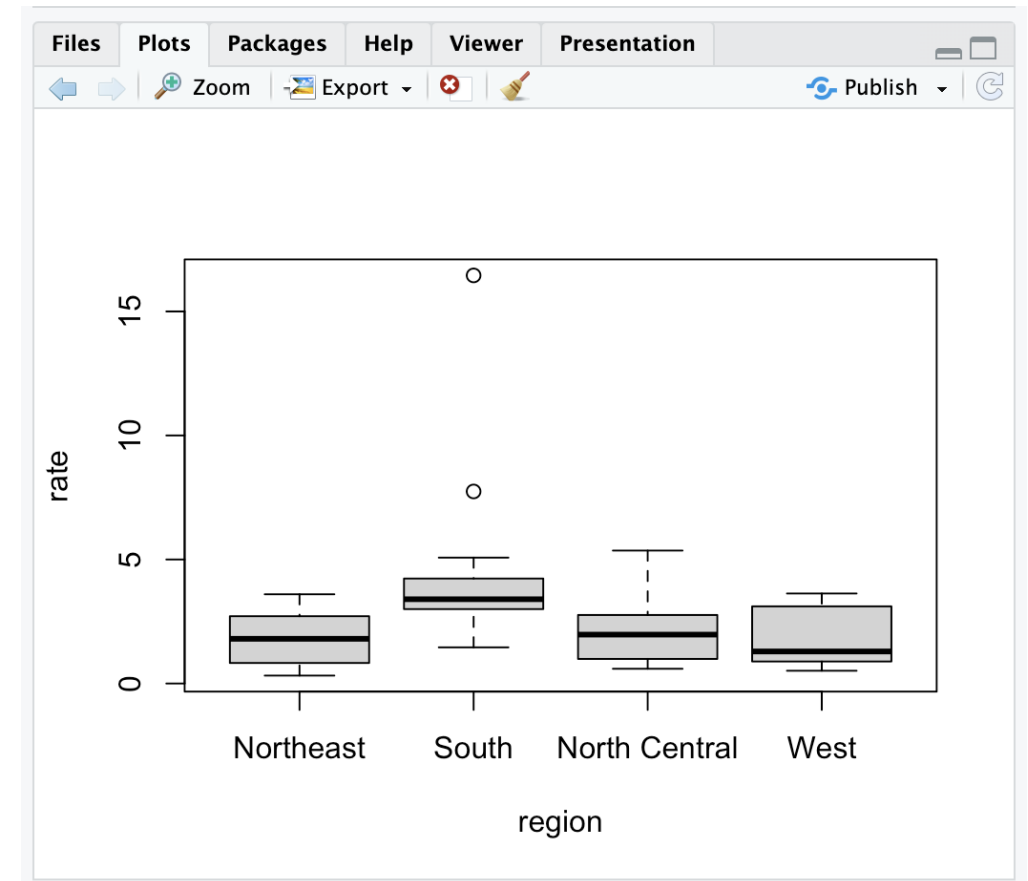
```
> x <- with(murders, total / population * 10e6)  
> hist(x)
```



Boxplot demonstrating the locality, spread and skewness groups of numerical data

R 4.2.2 · ~/

```
> murders$rate <- with(murders, total / population * 100000)  
> boxplot(rate~region, data = murders)
```



?~"

<https://stackoverflow.com/questions/14976331/use-of-tilde-in-r-programming-language>



In this lecture

- Introduction to R and RStudio
 - RStudio workspace
 - Environment setup, install and load packages: `tidyverse` `dslabs`
- Basics of R programming
 - Objects: `<-` (left assign), `=` (equal sign), environment panel
 - Built-in functions & operators: how to get documentation? Arguments?
 - Built-in datasets: `data()`
- Data types & data structures
 - Atomic data types: numeric, integer, character, logical, factor (`levels` and `reorder`)
 - Data structures: `data.frame`, vector, list, matrix
- Basic operations
 - Sorting: `sort`, `order`, `max`, `min`, `which.max`, `which.min`
 - Vector arithmetic: be aware of **recycling**
 - Indexing operators: `$`, `[]`, `[[[]]`
 - Indexing with integers, names, integer vectors, logical vectors
 - Helper functions to get indices: `which`, `match`, `%in%`